

SZÉCHENYI ISTVÁN EGYETEM
GÉPÉSZMÉRNÖKI, INFORMATIKAI ÉS VILLAMOSMÉRNÖKI KAR
INFORMATIKA TANSZÉK



SZAKDOLGOZAT

Hangfájl adatbázist adminisztráló webalkalmazás

Nyiri Zoltán

Mérnökinformatikus BSc

Győr, 2025

FELADATKIÍRÁS

Diplomamunka címe: Hangfájl adatbázist adminisztráló webalkalmazás

Hallgató neve: Nyiri Zoltán

Neptun-kód: BS6D9I

Szak: Mérnökinformaticus BSc

Tagozat: nappali

Kezdő tanév és félév: 2024/25 II.

Nyelv: magyar

Típus: nyilvános

Feladat:

Egy olyan webalkalmazás megtervezése és elkészítése, amely zenei hangfájlok kezelését biztosítja.

Részfeladatok:

a) A szakdolgozat elkészítéséhez szükséges résztevékenységek:

A megfelelő szakirodalom megismerése. A megoldás háttérében lévő adatbázis megtervezése és létrehozása. A szoftver főbb funkciói: regisztráció, hangfájlok kategorizálása, feltöltése, letöltése; hozzászólások, értékelések, jogosultságok kezelése.

b) A szakdolgozat főbb részei:

A zenekészítő szoftverek működésének rövid ismertetése. A feladatra megtervezett és elkészített programrendszer dokumentációja.

Belső konzulens: Pusztai Pál e. adjunktus, Matematika és Számítástudomány Tanszék

Külső konzulens: Rajkai Ádám óraadó, Széchenyi István Egyetem

Győr, 2024. 12. 18.



belső konzulens

tanszékvezető



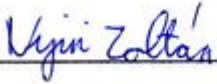
külső konzulens

Nyilatkozat

Alulírott, Nyiri Zoltán (BS6D9I) mérnökinformatikus Bsc. szakos hallgató kijelentem, hogy a „Hangfájl adatbázist adminisztráló webalkalmazás” című szakdolgozat feladat kidolgozása a saját munkám, abban csak a megjelölt forrásokat, és a megjelölt mértékben használtam fel, az idézés szabályainak megfelelően, a hivatkozások pontos megjelölésével.

Eredményeim saját munkán, számításokon, kutatáson, valós méréseken alapulnak, és a legjobb tudásom szerint hitelesek.

Győr, 2025. 12. 03.



hallgató

Konzultációs lap szakdolgozathoz és diplomamunkáhozHallgató adatai

Név: Nyíri Zoltán

Neptun-kód: BS6D9I

Szak: Mérnök informatikus BSc

Dátum	Tevékenység	Aláírás
2025.02.05	A feladat megbeszélése.	
2025.05.07	Az első félévben elvégzett munka értékelése, a munka dokumentálásának megbeszélése.	
2025.11.12	Az elkészült program megtekintése, a további teendők megbeszélése.	
2025.11.26	Az elkészült szakdolgozat dokumentációjának megbeszélése.	

A dolgozat a formai követelményeknek megfelel.

2025.12.03.

dátum



Puskai Pál egy. adj.

NYIRI ZOLTÁN

HANGFÁJL ADATBÁZIST ADMINISZTRÁLÓ

WEBALKALMAZÁS

ÖSSZEFOGLALÓ

Fő célom a modern zenekészítés világának bemutatása, működésének és felépítésének folyamata és a produceri munkafolyamat megkönnyítése volt. A megvalósításhoz Angular keretrendszert és NodeJS alapú futtatókörnyezetet használtam.

A szakdolgozatom egy olyan, a zenekészítéshez szükséges hangfájlokat kezelő és adminisztráló webalkalmazás, mely hazánkban teljesen egyedi megoldást kínál. A legfőbb cél – a megfelelő kezelhetőség és felhasználóbarát szemlélet megalkotása mellett – egy olyan rendszer megalkotása volt, mely nem csak egy átlagos zenehallgató számára nyújt kikapcsolódást és böngészést, hanem a hosszú, olykor bonyodalmas produceri és zenekészítői munkát hivatott lerövidíteni oly módon, hogy akár a saját zenéjébe teljesen beépíthető, vagy ahhoz ötletként és inspirációként szolgáló hangfájlok közül találjon számára megfelelőt.

A weboldal funkcionalitásából és összetettségéből adódóan számos új tapasztalatot szereztem egy ekkora projekt megtervezése és megvalósítása terén. A fejlesztés során adott működési elveket és funkciókat többször is újra terveztem a jövőbeli továbbfejlesztés és publikálás érdekében. Ennek főbb szükséglete az esetleges majdani kódátírások, a leegyszerűsítés és akár a folyamatos frissítések és modernizálások megvalósításának előkészítése a webalkalmazásban.

A tervezési dokumentáció tartalmazza azon sajátos megoldásokat, melyeket a fejlesztés során kiviteleztem. Részletezem a felhasználói és admin jogkörben szereplő interakciókat és folyamatokat, valamint a program otthoni környezetben történő elindításához szükséges lépéseket.

A weboldal megfelelő használatát felhasználói megközelítésből a részletes felhasználói dokumentáció tartalmazza.

Jövőbeli céljaim között szerepel a weboldal nyilvánosságra hozatala és továbbfejlesztése, valamint egy mobilalkalmazás létrehozása a felhasználóbarátabb szemlélet, a terjeszkedés, illetve a könnyebb elérhetőség érdekében.

Kulcsszavak: Windows, adatbázis-kezelés, hangfájl, producer, beat, loop

NYIRI ZOLTÁN

WEB APPLICATION FOR AUDIO FILE DATABASE MANAGEMENT

ABSTRAKT

The goal of my thesis was to present the world of modern music production, introduce its processes and structural foundations, and to simplify the workflow of music producers. For the implementation, I used the Angular framework along with a Node.js-based runtime environment.

My thesis focuses on developing a web application that manages and administers audio files used in music creation, offering a completely unique solution in Hungary. Besides achieving proper usability and a user-friendly approach, the emphasis was on creating a system that not only provides entertainment and browsing opportunities for the average music listener, but also helps shorten the often long and complex workflow of producers and music creators. The application enables users to discover audio files that can be fully integrated into their own compositions or serve as inspiration and ideas for future projects.

Due to the functionality and complexity of the website, I gained extensive experience in planning and implementing a project of this scale. Throughout development, I redesigned several operational principles and features to support future enhancements and public release. The main purpose of these redesigns was to prepare the web application for potential code refactoring, simplification, continuous updates, and modernization.

The technical documentation includes the specific solutions implemented during development. It details the interactions and processes available for both standard users and administrators, as well as the steps required to run the application in a local environment.

The proper usage of the website from a user-centred perspective is described in the comprehensive user documentation.

My future goals include making the website publicly available, further improving its features, and developing a mobile application to enhance user-friendliness, expand accessibility, and provide easier handling.

Keywords: Windows, database management, audio file, producer, beat, loop

TARTALOMJEGYZÉK

1. BEVEZETÉS	3
2. A ZENEI ALAP.....	6
2.1. Zenei alap főbb részei	6
2.1.1. Tempó.....	7
2.1.2. Dallamvezetés.....	8
2.1.3. Dobok	8
2.1.4. Basszus	10
2.2. A zenekészítő szoftverek rövid ismertetése.....	10
2.2.1. DAW szoftverek bemutatása.....	11
2.3. A webalkalmazás célja.....	13
3. TERVEZÉSI DOKUMENTÁCIÓ	14
3.1. Rendszerspecifikáció	14
3.2. Adatbázis terv.....	15
3.2.1. Adatbázis táblák és a köztük lévő kapcsolatok	15
3.2.2. Adatbázis leírása	16
3.3. Regisztráció és jogosultságkezelés.....	20
3.3.1. Regisztráció és a fiók megerősítése	20
3.3.2. Jogosultságkezelés	24
3.3.3. Jogosultság ellenőrzés és validáció futásidőben	25
3.4. Loop-ok feltöltése, megjelenítése és hozzászólás	27
3.4.1. Loop-ok feltöltése	27
3.4.2. Loop-ok megjelenítése	30
3.4.3. Hozzászólás	33
3.4.4. Like-olás.....	34
3.4.5. Letöltés	36
3.5. Kreditrendszer	39
3.5.1. Működés	39
3.5.2. Kreditszerzés és kreditvesztés	39
3.6. Üzenetküldés	42
3.6.1. Valós idejű chat modul.....	42
3.7. Profil oldal.....	46

3.8.	Admin jogosultság.....	49
3.8.1.	Admin menü	49
3.8.2.	Jelentések létrehozása, kezelése és elbírálása	51
3.8.3.	Loop-ok módosítása	54
3.8.4.	Új felhasználó loopjának publikálása.....	55
3.9.	Értesítések	56
3.9.1.	Értesítések létrehozása	56
3.10.	Tesztelés	59
3.11.	A webalkalmazás továbbfejlesztése	59
4.	FELHASZNÁLÓI DOKUMENTÁCIÓ.....	60
4.1.	Rendszerkövetelmény	60
4.2.	Szoftverkövetelmény.....	60
4.3.	Telepítés és elindítás	60
4.4.	Az alkalmazás használata.....	61
4.4.1.	Kezdőoldal	61
4.4.2.	Regisztráció és bejelentkezés	62
4.4.3.	Kreditrendszer	63
4.4.4.	Loop-ok	63
4.4.5.	Profil.....	66
5.	IRODALOMJEGYZÉK	67

1. BEVEZETÉS

A zenetörténet nem csak dallamok és ritmusok sorozata, hanem a hangszerek történelme – azoké az eszközöké, melyek az emberi lélek legérzékenyebb rezdüléseit formálják hanggá. Ezen hangszerek évszázadokon keresztül fejlődtek, magukban hordozzák az adott kor szellemét és technikai tudását. Szakdolgozatom bevezetőjében erről a csodálatos időutazásról szeretnék írni, a zene megalakulásától a digitális hangstúdiók világáig.

Az őskori ember életében már a mindennapok velejárója az éneklés és a kezdetleges hangszereken való játék. Hangszerek hiányában a hangját és a testét használta. A zene a legfontosabb szerepét a különféle rituálékban töltötte be. Zenéltek, amikor imádkoztak a bő termésért, gyógyításért, a sikeres vadászatért, vagy amikor ellenséggel kerültek szembe (ezek voltak a harci indulók vagy győzelmi énekek). Később kezdtek el természetes anyagokból hangszereket készíteni. A neandervölgyi ember szellemi életére fényt derítő felfedezések a nyugat-szlovéniai Divje Babe-i barlang lelőhelyen folytatott hosszú (1979–1999) ásatások eredményeként 1995-ben egy hangképzésre alkalmas tárgyat találtak, amelyről megállapították, hogy egy medve combcsontjából faragott furulya [1].

A zenének nagy jelentősége volt az ókori kultúrák művészetében is. Erről már különféle képzőművészeti alkotások és feljegyzések tanúskodnak. Platón görög filozófus szerint „... azért van olyan nagy fontossága a zenei nevelésnek, mert a ritmus és a dallam hatol be legjobban a lélek belsejébe, azt hatalmas erővel megragadják, s jó rendet hozva magukkal – azt, aki helyes elvek szerint nevelkedik, rendezett lelkű emberré teszik... A zene nem végződhetik másutt, mint a szép szeretetében” [2]. Püthagorasz (Kr. e. 570–495) már megkülönböztette a hangközök hosszát és arányát. A legenda szerint felfedezte, hogy a hangjegyek matematikai egyenletekre fordíthatók, amikor egy nap elhaladt a kovácsok mellett, és meghallotta a kalapácsuk üllőinek csattogását [3].

Már a 4. század környékén megjelentek a különféle zenei jelölések, melyek kezdetben a dallam hangzásbéli emelkedését vagy csökkenését jelezték. A dallamok hangzásvilágán keresztül megszületett a többszólamú¹ zene, amely új korszakot nyitott a zenei gondolkodásban. A középkor hangszerei még az egyszerűség és az egyházi istentisztelet eszközei voltak. A lant lágy pengetése vagy az orgona hangjátéka a kor szellemi atmoszféráját tükrözi. I. Gergely pápa elsőként rendszerezte a keresztény egyház teljes dallamkincsét és összeállította az akkor használatos énekek dallamgyűjteményét, ami lényegében a 15. század végéig változatlan maradt. A 15. század közepétől viszont érezhetően változott a hangzási eszmény. Általánossá vált a négyszólamúság, amely legtisztábban az énekhangok szólamrendjében érvényesült. A 16. század elején Európa-szerte felkelések jelzik a feudális rend válságát. Az a zenei stílus, melyet a középkor nagy mesterei, például Ambrus püspök vagy I. Gergely pápa felépítettek nem felelt meg többé az újkor elvárásainak. Az egyházi zene újításait fokozatosan átveszi a világi zene, és saját dallamokkal kiegészítve a vándormuzikusok művészete. A trubadúrok² kedvenc hangszere a lant, pengetése új dallamok felszínre törését hozza magával. A középkor végéhez

¹ Jelentése: egy zeneműben egyszerre több, önálló dallam szólal meg.

² Jelentése: az eszményi, rajongó szerelmet és vitézséget éneklő nemesi, udvari költő.

BEVEZETÉS

közeledve azonban egyre több művész, író, zeneszerző érzi úgy, hogy a görög és római kultúra újjáéled. Jóval szorosabb kapcsolat jön létre a zene és a szöveg között. A zeneszerzők legjobbjai arra törekednek, hogy világi darabjaik szövege értékes legyen, emiatt nagy hangsúlyt helyeztek a szövegek érthetőségére. A század vége fele a zenei játék egy újabb korszak, a barokk felé mutat [4].

Az opera műfajának hatalmas sikerei kezdik háttérbe szorítani a hangszeres műfajokat, a 19. században pedig szinte egyeduralkodó a zene területén. A barokk zene korát a tudósok az opera megszületésétől 1750-ig Johann Sebastian Bach haláláig tartják. A zenetörténetben a legsokoldalúbb korszak, ünnepélyes hangvétel, lenyűgöző arányok, hirtelen tempóváltások jellemzik. Megjelenik a hangszerelés, amikor a zeneszerző pontosan leírja, hogy melyik szólamot melyik hangszer játssza. Így alakul ki a zenekar. Szeparálódnak a magas- és mélyvonós hangszerek kezdetben 5, majd 4 szólamba (első és második hegedű, brácsa, gordonka, a második és az ötödik brácsa, vagy a tenorbrácsa) [5]. A játékot mély hangfekvésű hangszerek kísérik - mint a csembaló, az orgona, a lant vagy a fagott. Ekkorra fejlődnek ki a vonós hangszerek, amelyek kisebb-nagyobb változtatásokat leszámítva teljesen megegyeznek a mai változatokkal. Jean-Jacques Rousseau szerint a barokk zene az, melyben a harmóniák zavarosak, telezsúfolva modulációkkal³ és disszonanciákkal⁴; a dallam érdes és kevésbé természetes, az intonáció⁵ nehéz, a tempó erőltetett.

A 17. század végére a szenvedéllyel teli, rétegzett zenei stílus fokozatosan lecsendesedik, a hangzás kezd világosabbá, tisztábbá válni. A vízszintes szerkesztési elvet a függőleges váltja fel, azaz megszűnik a szólamok egyenrangúsága: egy dallam van, a többi ennek van alárendelve. E kor klasszikus tehetsége, akit mindenképpen meg kell említenem, Wolfgang Amadeus Mozart, akinek zenei stílusa ötvözte a 18. század második felének ízlésvilágát, miközben a műveibe fokozatosan építette be egyéni hangzásvilágát. Hangszerelésében a fúvósok (különösen a rézfúvósok) addig nem ismert hangszíneit fedezte fel, elsőik között alkalmazott például szordinált (hangfogóval tompított) trombitát, illetve klarinétot [6].

Az 1820-as években Liszt Ferenc (1811–1886) érzelmdús zongorajátéka nagy feltűnést keltett. Olyan dallamokat szólaltatott meg, amely új értelmezést adott a korszak muzsikájának. A romantikus zene törés nélkül alakult ki a klasszikus zenéből. A dallam a lelki kifejezés eszköze lett. Minden eszköz, struktúra, forma, játéktechnika, hangzás kibővült. A zenekar mérete jelentősen megnőtt az előző század gyakorlatához képest. A zeneszerzők új hangszerelési effektusokkal gazdagították a zenekari hangzást. Az addig alkalmazott hangszerekből több szerepelt egy-egy szólamban. Különösen a fúvós és ütős csoportok szólamát növelték meg és új ütőhangszerek alkalmazására is sor került [7].

Ludwig van Beethoven IX. szimfóniájában már nagydobot, cintányért és triangulumot is alkalmaz, Hector Berlioz csőharangot, összeütős cintányért, függesztett cintányért, tenordobot és nagydobot is ír Fantasztikus szimfóniájába [8].

³ moduláció: Hangnemváltozás.

⁴ disszonancia: Hamis hangzás.

⁵ intonáció: Hangvezetés.

BEVEZETÉS

A rézfűvós hangzásideál kitágult, ami hihetetlen hangszerépítési hullámot indított el. Például Berlioz, Wagner vagy Mahler hangszínkeresése új hangszerek (basszustrombita, Wagner-tuba, kontrabasszus harsona, basszustuba, új hangszertechnikai elemek, különböző billentyűzetek, szordínók) zenekarba emelését eredményezte [9].

Ugyanezen időszakon belül az Egyesült Államokba majd félmillió rabszolga érkezett, akik magukkal hozták zenei hagyományait. Ezt követően folyamatosan jelennek meg a korai dzsesszes hangzások népszerű szabadtéri vagy táncos összejövetelek zenéjében és a katonai rezesbandák hangzásában, amely az 1950-es évekre a rock and roll korszakát építik fel. Az itt használt rézfűvósok és ütőhangszerek adják meg ma is a dzsessz alaphangzását. A rádió és a professzionális hanglemezzelgyártás az 1920-as években terjedt el, ezek erősen hatottak a dzsessz és a későbbi rockzene terjedésére.

A 1960-as évek vége felé a rockzenészek már elektronikus hangszereket kezdenek használni, mint például a theremin és a mellotron, hogy finomítsák és egyben határozottabbá tegyék hangzásukat. Ahogy a világ egy új korszakba lép, az elektronikus rock, a szintipop és az újhullámos zenekarok hulláma jelenik meg, örökre megváltoztatva az addig megszokott populáris zenei stílust. Új zenei formák kísérletezése válik a műfaj középpontjává. Az Egyesült Királyságban és Németországban megszületik a progresszív rockzene, amely a billentyűzetek és a szintetizátorok mellett – a rockzene számára idegennek vélt – hangszerek számára is biztosít lehetőséget. Gyakran jelenik meg szaxofon, hegedű, trombita vagy vibrafon a dalokban (lásd: Yes, Pink Floyd, Genesis vagy Dire Straits).

A 80-as évek vége felé a szintetizátor egyre nagyobb teret nyer a könnyűzene világában és aktív használata átmenetet képez a progresszív rock és az elektronikus rock, illetve elektronikus zene között. A mai elektronikus zene a '70-es évekhez képest megváltozott.

Manapság a zenekészítés több nagy témakört magába foglaló kifejezés. Szerepelnek benne az elmúlt korszakok jellegzetességei, hangszerelései, dallamai és a számítástechnika által nyújtott új lehetőségek egyaránt, melyek a mai zenei stílusok egyediségét adják. A hangszerek összeállításához és a hangszereléshez már különböző szoftverek állnak rendelkezésre, melyek használatához a zeneelméleti és zeneírási tudás elengedhetetlen. Szakdolgozatom célja ezen zenei paletta felépítéshez, megtervezéséhez és hangszereléséhez nyújt segítséget. Segíti a zenekészítők közötti együttműködést, valamint szabadon felhasználható hangsávokat tartalmaz, melyek segítségével továbbfejleszthető a zenei projekt.

A továbbiakban kifejtem, hogy pontosan miről szól a zenekészítés világa, hogyan készülnek ma a zenék és milyen tényezők együttes hatása szükséges ahhoz, hogy egy sláger alapzenéje életre keljen.

2. A ZENEI ALAP

Egy zene eladhatóságához az egyik legnagyobb szereplő az előadó. Az a személy, aki odaáll a mikrofon elé, aki kiáll a színpadra. Hisz az ő hangját halljuk a zenében, emiatt – egyébként joggal – gondolhatjuk, hogy övé az érdem. Azonban ez nem feltétlenül igaz – legalábbis nem olyan formában, mint gondolnánk. Olyannyira nem, hogy a továbbiakban tekintsünk is el önmagukban az előadóktól. Hisz ők gyakran „csupán” a hangjukat adják a produkciókhoz. De mi is kell még egy kész zenéhez?

Itt hoznám szóba szakdolgozatom témájának egyik kulcsemberét. Azt a személyt, aki a zenei alapot – sok esetben az előadóval együtt – megálmodja, majd megvalósítja. A továbbiakban erre a személyre, mint *producer*-re fogok hivatkozni.

A produceri munka a zenekészítés nulladik fázisa. Mondhatni az első dolog, melynek a gördülékeny munka érdekében akárcsak vázlagszerűen, de valamilyen formában el kell készülnie. Ezt nevezzük zenei vázlatnak, másnéven *demo*-nak. Ritkább esetekben ezt a vázlatot van, hogy maga az előadó készíti el, valamint akadnak olyan előadók is, akik a teljes zenei alapot maguk írják – ez természetesen azt jelenti, hogy ők saját műveikben nemcsak mint előadók, hanem mint producerek is tevékenykednek.

2.1. Zenei alap főbb részei

Ahhoz, hogy egy dal megragadja a hallgatót, rengeteg tényezőnek kell benne illeszkednie. A hangsúly mindig az előadón van, azonban a háttérben minden zenei motívumnak pontosan passzolnia kell egymáshoz. Minden kisebb-nagyobb elemnek összhangban kell lennie, és ezek összessége adja azt a zenei alapot, melyre később a dalszöveg kerül.

A zene stílusa – mint azt már korábban részleteztem – már önmagában is rengeteg kategóriába sorolható, melyekről csupán a nevük hallatán sejthetjük, hogy milyenfajta dallamok fognak felcsendülni. Ilyen például a klasszikus zene, a country, a jazz, és ide sorolandó a manapság egyre gyakoribb pop, hiphop vagy rap kategóriájú alapzenék⁶. Ezen kategóriákat is még tovább lehetne darabolni, ám a lényeg, hogy mindezt a zenében lévő motívumok határolják be. Többségük ugyanolyan elemekből áll össze, azonban más formában, és ez adja a zenei műfajok közti különbséget.

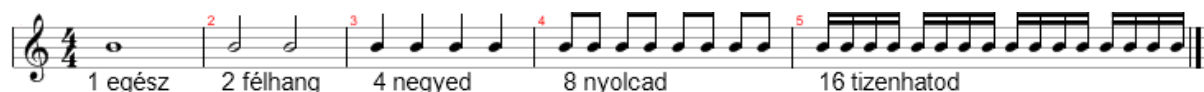
A következőkben azon zenei elemeket fogom részletezni, mely az alapzenék többségében megtalálható, befolyásoló tényezője van a dal stílusának megnevezésében és melyek szakdolgozatom fő témaadójaként szolgálnak.

⁶ alapzene: Az a hangsáv, amely az instrumentális alapot jelenti, vagyis a zenei motívumokat és elemeket tartalmazza, de a vokált és az előadó hangsávját nem.

2.1.1. Tempó

A tempó, másnéven ütésszám vagy BPM⁷ egy dal gyorsaságát, ritmusát adja meg. Ez egy számban értendő fogalom, melyet nem csak zenekészítésben, de az orvostudományban is használnak – a pulzusszámot is ez alapján a fogalom alapján határolják be. (Az emberi szív például átlagosan 72-80 BPM-ben dobog.)

A producereknek a BPM azt jelenti, hogy egy perc alatt mennyi negyedhang hangzik el a zenében. Ennek megértéséhez tisztában kell lennünk a hangokkal, a hangok hosszával és elrendezésével (1. ábra).



1. ábra. Ütemek, hangjegyek és hangjegytypusok

A fenti ábrán egy 4/4-es ütemmutatóval jelzett kottarészlet látható. A sor elején jelölve van, hogy egy ütemben hány negyedhang szerepelhet. Ezt szigorúan be kell tartani, tehát nem hozható létre egy ütem úgy, hogy abban ne pontosan annyi negyedhangnak megfelelő érték legyen, mint amennyi a legelején meg van határozva. A zenében a leghosszabb hang az *egész*. A sorrendben minden ezt követő hangjegy értéke pontosan fele az előző értékének: az *egész* után a következő leghosszabb hangterjedelmű jegy a *félhang*, ezt követi a *negyed*, majd a *nyolcad*, a *tizenhatod* és így tovább. Ezáltal egy 4/4-es ütemben pontosan egy darab egész, két félhang, négy negyed – és így tovább a hangjegyek értékei szerint – található. Az első ütem kezdete nincs jelölve, azonban minden esetben egy függőleges vonallal vannak elválasztva egymástól. A vízszintes, egész sort végig vezető vonalak arra szolgálnak, hogy a rajtuk megfelelően elhelyezett hangjegyet pontosan be lehessen határolni és azt később lejátszani.

Minden zenének van BPM-je, melyek stílusuktól függően változhatnak, azonban nem ritka az sem, hogy ez a szám több olyan mű esetén is megegyezik, melyek más kategóriákba tartoznak.

⁷ Beats Per Minute

1. táblázat. Főbb zenei stílusok jellemzői

Zenei stílus	Jellemző BPM-tartomány	Megjegyzés
Trap	120–160 BPM	Gyors, feszes, energikus; sokszor 140 BPM körül a legjellemzőbb.
Pop	100–130 BPM	Közepes tempójú, könnyen fogyasztható, fülbemászó dallamok.
HipHop	80–110 BPM	Lassabb tempó, inkább a szöveg és a ritmika hangsúlyos.
Rock	110–160 BPM	Széles skálán mozog: a klasszikus rock 120 BPM körül, a punk/metal stílusok pedig 150 BPM körül.
Jazz	90–160 BPM	Nagyon változatos, lassabb tempóktól a gyorsabbakig egyaránt.

2.1.2. Dallamvezetés

A dallam vagy melódia (*melody*) a zene egyik legfontosabb alappillére. Abból, hogy milyen hangokon fekszik és milyen hangokból épül fel pontosan megállapítható a mű hangneme⁸. Befolyásolja a dal hangulatát, visszatérő elemként szolgál egyes szakaszokban (például gyakran teljesen ugyanaz a dallam egy zene minden refrénjében), valamint egyedisége gyakran változik a kész zene egyes részein. Azonban az esetek túlnyomó többségében a dallam által beállított hangnem változatlan marad, hisz erre épülnek fel később a különböző hangszerek játéka.

Gyakori eset, hogy egy zene fődallama mellett több kísérő dallamot is írnak hozzá. Ez általában halkabb, sokszor alig észrevehető hangokat jelent a zenében, melynek feladata a szebb zenei összhang megteremtése [10][11].

2.1.3. Dobok

A dob, mint zenei elem a legsokoldalúbb és sokszínűbb eszköz a zene felépítésében. A modern zenék esetében például csupán a dob motívumaiból megállapítható a zene műfaja és kategóriája. Hiába egy nagy kategória, de mégis annyi eszközt ölel magába, hogy kénytelenek vagyunk ezt is több részre bontani [12].

1. Hihat sets



A lábcin (hihat sets, hihats) a dobkészlet egyik elengedhetetlen eleme. Hagyományos szerelésében két, egymással szembenéző cinből áll (2. ábra), melyeket pedállal és ütővel is meg lehet szólaltatni.

2. ábra. Lábcin

⁸ hangnem: Egy dallamban a hangjegyek egymáshoz való kapcsolódási viszonya, melyből a zene épül.

Ezt a modern zenében már különböző szoftverekben használható hangfájlokkal készítik, ügyelve arra, hogy egy zenekar esetén is közel ugyanúgy el lehessen játszani, mint ahogy a zenében hallatszik. Feladata leggyakrabban a zenei stíluselemek megadásának segítése – ezt az egyes megszólaltatások időzítése és hossza tudja befolyásolni –, valamint a tempó folyamatos tartása.

2. Snare és Clap



A snare az élő dobkészletben a pergődobnak (3. ábra) felel meg. Általában az ütem elején, vagy a háromnegyedénél használják, ezzel megadva a kezdőlöketet egy adott ütem zenei motívumainak, valamint díszíti és színesíti a zene struktúráját, használatuk azonban itt is a zene stílusától függ.

3. ábra. Pergődob

A clap viszont különbözik az előzőektől. Jelentése *taps*, s ez a zenében is egy tapshoz hasonló hangeffektet jelent. Manapság szinte minden zenében megtalálható, általában a zenei stílustól függően az ütemrendszerek közepén helyezkedik el.

3. Lábdob

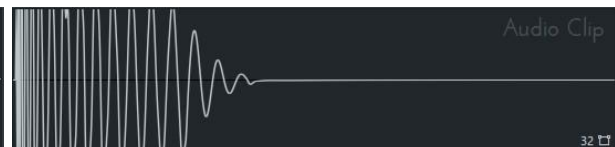


4. ábra. Lábdob

A lábdob (*kick*) a dobszerkezet legmélyebb, legütősebb ritmikai eleme. Fő feladata a tempó és a dinamika vezetése. A hagyományos dobfelszerelésen ez egy pedállal felszerelt nagydob (4. ábra), mely a modern zenében sem sokban változik. A különbség leginkább az erősségében és a lecsengésében mutatkozik meg - vannak lágy lábdobok (5. ábra), és ütős, erős lábdobok (6. ábra), melyeket úgyszintén a zenei szükségletek válogatják. Régebbi zenékben főleg az ütemek és ütemrendszerek első hangjaként szolgált, manapság ez már gyakran megjelenik az ütemben is szétszórva, a ritmikát továbbfejlesztő hangeszközként.



5. ábra. Gyenge lábdob hangsávja FL Studio-ban



6. ábra. Erős lábdob hangsávja FL Studio-ban

2.1.4. Basszus

Az alapzene egyik elengedhetetlen eleme a basszus. A zeneipar korábbi szakaszában ezt tipikusan basszusgitár játszotta, manapság ez már egy kicsit megváltozott. Bár színpadokon a zenekarok általában még mindig ehhez az eszközhöz nyúlnak, a zenekészítés folyamatában már ritkábban tűnik fel a gitár, mint a zene legmélyebb vonalát adó eszköz.

A modern zenei szükségletek kielégítéséhez nem elég egy megszokott, rövid ideig hallható basszusgitár a zene kulcseszközeként. Van, hogy erősebbnek kell hatnia, máskor gyengébbnek; szükségeselek lehetnek akár zenei átvezetések egyik hangról a másikra nagyon rövid időn belül - ezáltal a basszust, mint hangsávot a producerek már gyakran a szoftverek által nyújtott eszközök segítségével készítik el. Gyakori a szintetizátor a megfelelő effektek és eszközök hozzácsatolásával, valamint a más producerek által elkészített folyamatos, mély hangzású hangsávok, melyeket, mint kész zenei réteg lehet felhasználni és módosítani.

Zenei kategóriáktól függően kétféle basszusról beszélhetünk:

- Bassline
- 808⁹ Bassline

A Bassline tulajdonképpen az a fajta basszustípus, melyek leginkább az általános basszusgitár-hangzáshoz kötődnek. Folyamatosak, nincs bennük ütemvezető löket, csupán maga a mély hang, mely a zenét kíséri.

Mindemellett a 808 Bassline több dologban is eltér az előbbitől. Legfeltűnőbb különbség, hogy gyakran jelenik meg magában a basszusban is a lábdob. Zenei szükséglettől függően erősebben vagy gyengébben, de ez a fajta basszus a kick mellett – sok esetben *helyett* – az ütem- és dinamikavezetés fő elemeként szolgál. Egy hangsáv itt jellemzően hangosabb és rövidebb ideig tart, emiatt ezt a fajta basszust leginkább olyan zenékben szokták alkalmazni, melyben a dallam nagy hangterjedelemmel rendelkezik, vagy dinamikus, tempós zenei alap készül.[12]

2.2. A zenekészítő szoftverek rövid ismertetése

A modern zenekészítés világának működése szinte elképzelhetetlen a különféle szoftverek, úgynevezett DAW¹⁰-ok nélkül. Ezen szoftverek lehetővé teszik a többsávós hangfelvételt, hangsávok írását különböző – beépített vagy felhasznált – zenei hangszerek és eszközök segítségével, ezen fájlok szerkesztését, effektelését, valamint a hangsávok exportálását – akár egyesével, akár többet is egyszerre vagy a készterméket egyaránt.

Ezen szoftverek úgy működnek, hogy a producer hangsávokat készít, vagy hangfájlokat importál be a programba. A leggyakoribb hangfájl-formátumok, melyek veszteségmentes hangminőséget biztosítanak, a WAV¹¹ és az AIFF¹² formátum. A programokba azonban beilleszthető MP3¹³ kiterjesztésű fájl is, ennek hátránya azonban, hogy exportálás után

⁹ ejtsd: eight-o-eight

¹⁰ Digital Audio Workstation, zenefejlesztő környezet

¹¹ Waveform Audio File Format

¹² Audio Interchange File Format

¹³ MPEG Audio Layer III

veszteségessé válik, emiatt ezt a formátumot – a WAV fájlhoz képest kisebb méretének köszönhetően – főleg vázlatgeneráláshoz szokták alkalmazni.

2.2.1. DAW szoftverek bemutatása

A következőkben ismertetek három zenekészítő szoftvert, melyek a zenészek és producerek körében a legelterjedtebbek.

- **FL Studio [13]**

Az FL Studio (7. ábra) az egyik legkönnyebben használható, és a legelterjedtebb zenekészítő szoftver. Kezelése és könnyű átláthatósága miatt leginkább a kezdő zenekészítők körében terjedt el, azonban nem ritka, hogy világszinten is elismert és híres producerek is ezzel a szoftverrel dolgoznak – például Martin Garrix (*holland zenei producer*) vagy a 2018-ban elhunyt Avicii (*svéd zenész és zenei producer*).

- **Ableton Live [14]**

Az Ableton Live (8. ábra) az FL Studioval karöltve lefedik a Windows operációs rendszeren a leggyakrabban használt DAW-ok nagy százalékát. Csakúgy, mint az előbb említett szoftver, professzionális körülményeket és eszközöket biztosít bármilyen típusú zene elkészítéséhez. Különlegessége, hogy támogatja az élő előadások alatt történő esetleges hangsáv-keveréseket és módosításokat, melynek használata leginkább az élő zenét keverő DJ-k körében elengedhetetlen.

- **Logic Pro X [15]**

A Logic Pro X (9. ábra) egy főként MacOS¹⁴ rendszeren elterjedt DAW. Ezzel a szoftverrel nem csak zenei producerek dolgoznak – kiemelkedően magas hangszínekészletének, valamint a rengeteg beépített hangsávmódosításra szolgáló eszközeinek hála, filmzenék, hangeffektek, valamint más elektronikus műfajok készítésére is előszeretettel használják.

¹⁴ Az Apple által fejlesztett operációs rendszer

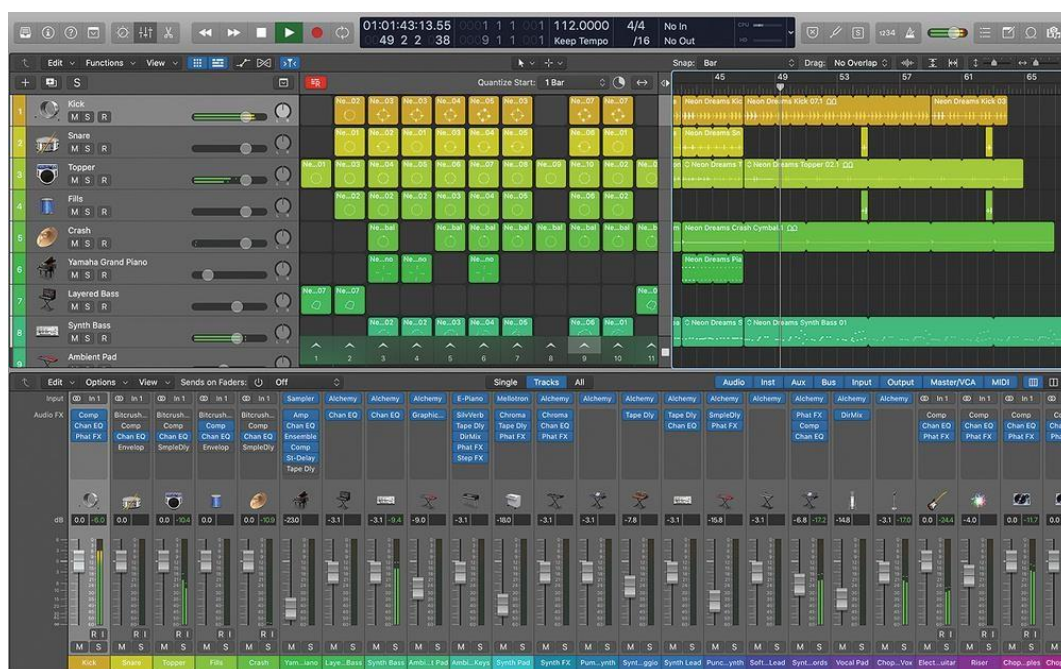
ZENEI ALAP



7. ábra. FL Studio szoftver



8. ábra. Ableton Live szoftver



9. ábra. Logic Pro X szoftver

2.3. A webalkalmazás célja

A zenekészítés folyamata során azonban gyakori, hogy a producerek egy adott szakaszon megrekednek. Ennek oka lehet többek között inspirációhiány, vagy csak új ötleteket szeretnének belevinni a zenébe, viszont nem tudják, mivel lehetne a projektet színesíteni. Ilyen helyzetben hatalmas segítséget jelenthet egy olyan platform, melyen mások által feltöltött hangfájlok közül böngészhetnek, meríthetnek ötleteket, valamint felhasználhatják saját zenéikben azt, amely épp megtetszett. Szakdolgozatom erre szolgál megoldásként; egy olyan webalkalmazás elkészítése, ahová bármely producer feltöltheti saját hangfájljait – többek között a már említett kategóriákban –, valamint mások letölthetik, és felhasználhatják azt. Mindezek mellett egyszerűsíti egyes producerek közti együttműködést és kapcsolatfelvételt.

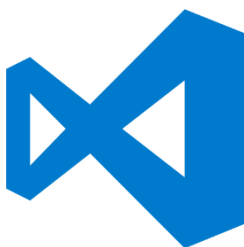
3. TERVEZÉSI DOKUMENTÁCIÓ

3.1. Rendszerspecifikáció



10. ábra.
Mongodb logó.

A webalkalmazást elsősorban zenei producerek, zenei előadók vagy zenekészítő szoftvert használó személy fogja használni. A működéshez szükséges adatsokaságot egy távoli szerveren futó MongoDB [16] adatbázisban tárolom.

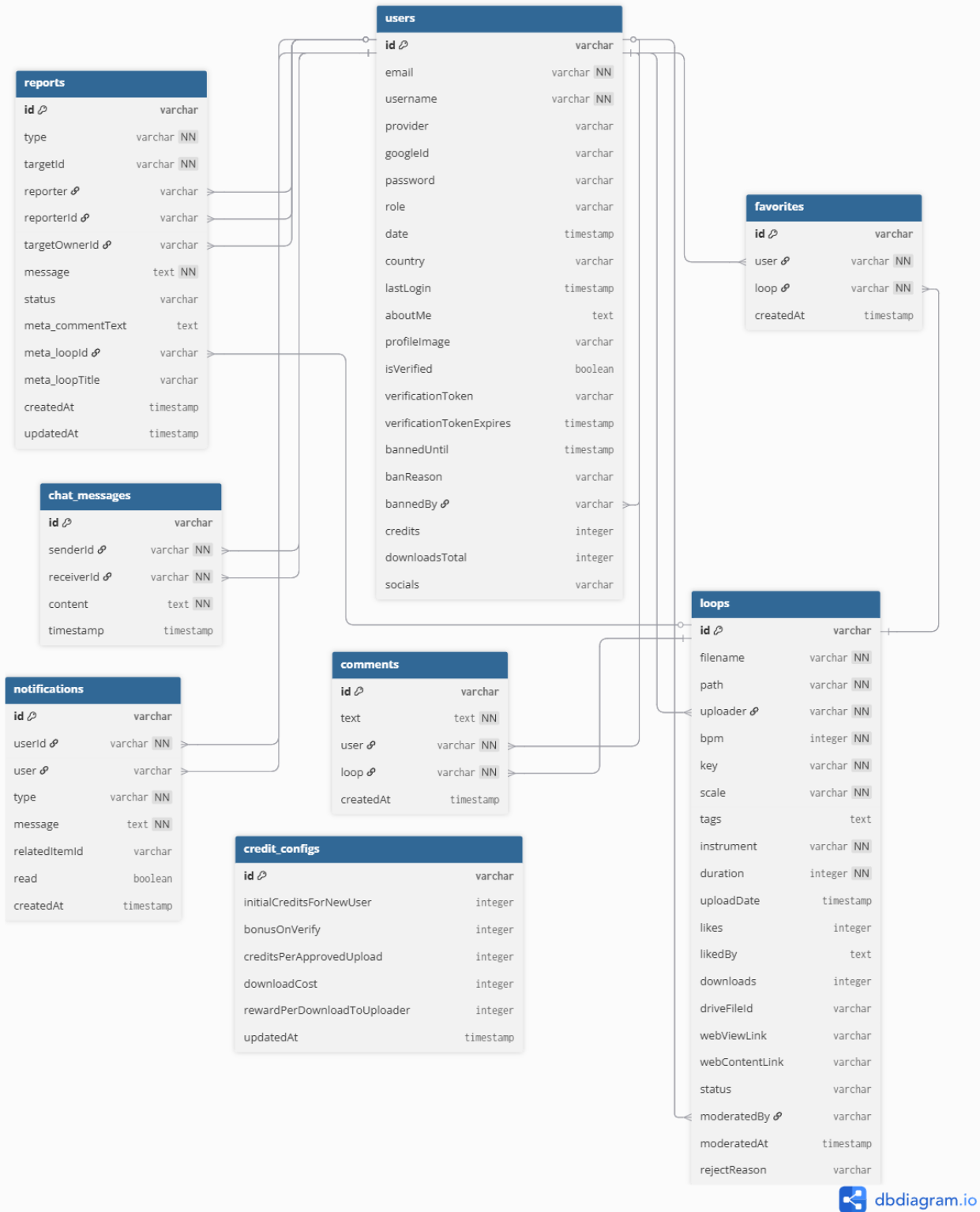


11. ábra.
Visual Studio Code logó.

A webalkalmazást Windows 10 operációs rendszeren fejlesztettem, a Visual Studio Code szoftverrel. A frontend fejlesztéséhez az Angular keretrendszer 19.1.0 verzióját használom. Ennek a megfelelő telepítéséről és beállításáról részletes leírást találunk a Visual Studio Code weboldalon [17]. A backend fejlesztése során a Node.js futtatókörnyezetet és az Express.js keretrendszert használom, melyek megfelelő implementálásáról a Node.js [18] és a Visual Studio Code weboldalon [19] található leírás. A projekt során a verziókezelést a Git nevű verziókövető rendszer segítségével oldottam meg.

3.2. Adatbázis terv

3.2.1. Adatbázis táblák és a köztük lévő kapcsolatok



12. ábra Kibővített egyed-kapcsolat modell (EER).

3.2.2. Adatbázis leírása

Az adatbázis kialakulásának menetében első fontos lépésként feltérképeztem azon elérhető szolgáltatásokat, melyek funkcióiban – bár eltérő célközönséget szolgálnak ki – hasonlóképp működnek. Minden táblában található egy egyedi azonosító, mely az adott tábla elsődleges kulcsa. A MongoDB különlegessége, hogy amennyiben nem állítjuk be ezen értékeket manuálisan, az adatbázisban létrehoz egy `ObjectID`-t, mely automatikusan a tábla elsődleges kulcsa lesz. Ez egy 24 hexadecimális karakterből álló egyedi azonosító, melynek felépítése a következőkből áll:

- Első 8 karakter: 4 bájtos időbélyegző (az azonosító keletkezésének időpontja)
- Következő 12 karakter: a gép MAC címéből és a folyamat azonosítóból generált sorozat
- Az utolsó 4 karakter: növekvő számláló arra az esetre, ha ugyanabban a másodpercben több `ObjectID` készült

2. táblázat. Ikonok és jelentésük.

Ikon	Jelentés	Példa
	Elsődleges kulcs (Primary Key)	id
	Foreign key (idegen kulcs, kapcsolat)	user, loop, uploader

A **Users** tábla tárol minden olyan adatot, mely a felhasználó fiókjához köthető. Itt szerepelnek a bejelentkezési adatok, a fiók állapota, tevékenységek, jogosultságok és összesített eredmények.

A **Loops** táblában található a felhasználó által feltöltött hangfájlok részletes adatai, mint például a tempót diktáló bpm, a hangnemet megadó key vagy egy új felhasználó esetén a loop státusza, valamint olyan integer típusú mezők mint a letöltés- vagy kedvelésszámláló.

A Loops táblához szorosan kapcsolódó **Comments** tábla mindössze öt mezőből áll. Ezen adathalmaz a részletes loop nézetben érhető el a hangfájl alatt, ahová a verifikált felhasználók megjegyzéseket írhatnak. Ezekhez szükséges a minden táblában megtalálható id, mint egyedi azonosító. Ezen felül egy text mező, mely a megjegyzés tartalma, user mező, mely meghatározza a megjegyzés íróját, egy loop mező mely a megfelelő hangsáv alá rögzíti a kommentet, valamint egy `createdAt` mező, mely a megjegyzés létrehozását hivatott elmenteni.

Úgyszintén összefügg a Loops táblával a **Favorites** tábla, mely – a nevéből adódóan – lehetőséget biztosít elmenteni azokat a hangfájlokat, melyeket később ismét meg szeretnénk az oldalon tekinteni. Ahhoz, hogy bekerüljön egy loop a kedvencek közé mindösszesen a felhasználó és a hangfájl egyedi azonosítójára van szükségünk.

A kategóriában szereplő hangfájllhoz új azonosítót rendelünk (ami eltér a loop egyedi azonosítójától a könnyebb kezelhetőségre való tekintettel), valamint itt is megtalálható a `createdAt` mező, mely a loop kedvencekbe való bekerülésének dátumát tartalmazza.

A hangfájloktól kicsit elkanyarodva a webalkalmazás megfelelő működésének megteremtése érdekében nem csak felhasználói, hanem egy nagyobb jogosultságokkal rendelkező admin környezetet is kialakítottam. Bár mint azt már említettem, ez a jogosultság és ezáltal annak hatóköre a User táblában van tárolva, a következőkben olyan táblákról is szó esik, mely közvetve vagy közvetlenül, de valamilyen módon kapcsolódik az admin jogkörhöz.

A nem megfelelő tartalmak elkerülése, a weboldal moderálása és az egyéb, más módon bántó felhasználók vagy tartalmak korlátozása és törlése miatt létrehoztam egy **Report**, avagy Jelentés rendszert. A jelentéseket a felhasználók tudják létrehozni, a magasabb jogkörrel rendelkezőknek ez az opció nem lehetséges. Jelenteni kommenteket, felhasználókat és loop-okat lehet, amik egy, csak adminoknak elérhető végpontra futnak be. A jelentés folyamata alatt a felhasználónak egy rövid üzenetben jeleznie kell a problémát az általa jelenteni kívánt tartalommal szemben, amely a `message` mezőben tárolódik, valamint a jelentett felhasználó vagy tartalom létrehozójának egyedi azonosítója, illetve a report-ot küldő felhasználó és annak létrehozási dátuma is mentésre kerül. Ennek bővebb működéséről a későbbiekben lesz szó.

Az alábbi programrészletben a Report tábla látható.

```
export type ReportType = 'comment' | 'loop' | 'profile';
export type ReportStatus = 'pending' | 'resolved' | 'rejected';

const ReportSchema = new Schema(
  {
    type: {
      type: String,
      enum: ['comment', 'loop', 'profile'],
      required: true,
      index: true
    },
    targetId: {
      type: Schema.Types.ObjectId,
      required: true,
      index: true
    },
    reporter: {
      type: Schema.Types.ObjectId,
      ref: 'User'
    },
    reporterId: {
```


TERVEZÉSI DOKUMENTÁCIÓ

```
    type: Schema.Types.ObjectId,
    ref: 'User'
  },
  targetOwnerId: {
    type: Schema.Types.ObjectId,
    ref: 'User'
  },
  message: {
    type: String,
    required: true,
    trim: true,
    maxlength: 2000
  },
  status: {
    type: String,
    enum: ['pending', 'resolved', 'rejected'],
    default: 'pending',
    index: true
  },
  meta: {
    commentText: { type: String },
    loopId:       { type: Schema.Types.ObjectId, ref: 'Loop' },
    loopTitle:    { type: String }
  }
},
{ timestamps: true, strict: true }
);
```

Annak érdekében, hogy a felhasználók naprakész információkat kapjanak minden olyan tartalomról, melyeket ők töltöttek fel vagy osztottak meg a weboldalon, egy értesítés panelt vezettem be, mely egy könnyen elérhető, folyamatosan frissülő lista a tartalomgyártóknak. Ehhez szükséges volt a **Notifications** tábla létrehozása az adatbázisban. E tábla és a Users kollekció userId mezője idegen kulcsként van összekapcsolva az alkalmazásban, hisz minden értesítés egy konkrét felhasználónak van címezve. A panel egy feltöltött loop belike-olása, letöltése vagy egy esetleges admin általi módosítása esetén, a feltöltött loop alatt egy hozzászólás keletkezése esetén, valamint a felhasználó első 5 loopjának feltöltése utáni manuális elbírálásának eredményéről küld értesítést. Ezek működését és logikáját a későbbiekben hosszasan kifejtem. A Notifications tábla mindezek mellett tárolja a külön értesítésfajtákhoz csatolt szöveget és a keletkezés dátumát az adatbázisban.

Az interneten elérhető hasonló weboldalakon hasonlóképp van elkészítve a kapcsolatfelvétel: ha egy felhasználó által feltöltött loop elnyeri valaki tetszését, és egy esetleges közreműködésre invitálná producerpartnerét, emailt kell írnia a másik félnek, ráadásul sok esetben ezt a weboldal nem is teszi lehetővé. Sokszor a weboldali emailküldés az

egyetlen módja a kapcsolatfelvételnek, ami lassúvá és bonyodalmassá teszi a közös munka kezdetét, valamint a közvetett üzenetváltások sem sürgetik meg a munka menetét. Ezért szakdolgozatomban erre egy sokkal egyszerűbb megoldást készítettem, a weboldalon keresztül történő üzenetküldéshez létrehoztam a **Chat** táblát az adatbázisban. Működése hasonlít a már jól ismert chateléshez: a felhasználó egy másik profiljára kattintva megtalál egy Üzenet küldése gombot, mellyel megnyílik egy chatfelület, ahol könnyen és gyorsan tud üzenetet küldeni a másik félnek. A chatelésnek egy külön üzenet ikonnal ellátott értesítőpanelja van, ahol az előbb leírt Notifications táblától és annak listájától különálló értesítést kap, melyre rákattintva nála is megnyílik a chatablak. Az adatbázis az üzenet tartalmán kívül a feladó és a címzett azonosítóját és az üzenetküldés időpontját rögzíti.

Az alábbi kódrészlet a Chat táblát tartalmazza:

```
const ChatMessageSchema = new Schema<IChatMessage>({
  senderId: {
    type: String,
    required: true
  },
  receiverId:
  { type: String,
    required: true
  },
  content: {
    type: String,
    required: true
  },
  timestamp: {
    type: Date,
    default: Date.now
  }
});
```

Adatbázisom utolsó táblája a weboldal egy hasznos és mindent körülölelő beállítását és logikáját foglalja magába **creditConfig** név alatt. Ezen tábla felelős a felhasználókhöz köthető kreditrendszer kialakulásáért és megfelelő működéséért, melyről részletesebben is fogok írni. Kreditet kaphatnak a user-ek egy loop feltöltéséért, azért is, ha ezt a loop-ot mások letöltik, valamint a beregisztrálásért is. Azonban veszítenek, ha mások által feltöltött loop-okat töltenek le. Ezen beállítások módosíthatók az admin felületen. A tábla mindezek mellett az utolsó módosítás időpontját is tárolja.

Kódrészlet a creditConfig táblából:

```
const CreditConfigSchema = new Schema<ICreditConfig>({
  initialCreditsForNewUser: {
    type: Number,
    default: 0
  },
  bonusOnVerify: {
    type: Number, default: 2
  },
  creditsPerApprovedUpload: {
    type: Number,
```

```

        default: 2
      },
      downloadCost: {
        type: Number,
        default: 1
      },
      rewardPerDownloadToUploader: {
        type: Number,
        default: 0
      },
      updatedAt: {
        type: Date,
        default: Date.now
      },
    },
  });

```

3.3. Regisztráció és jogosultságkezelés

A webalkalmazás két, egymástól különálló jogosultsági szintet különböztet meg. Alapesetben a beregisztrált felhasználók *user* szerepkör, valamint az *admin* szerepkör. E szerepkör a már részletezett **Users** kollekcióban kerül tárolásra a *role* mezőben. A szerepkör határozza meg, hogy egy adott felhasználó mely végpontokhoz férhet hozzá, milyen jogosultságai vannak és mely funkciók azok, amik tiltottak a számára.

3.3.1. Regisztráció és a fiók megerősítése

Regisztrálás után a felhasználó automatikusan megkapja a *user* szerepkört. Azonban a regisztrálástól függően, e szerepkör még korlátozott jogosultságokkal rendelkezhet: a felhasználó regisztrálhat adatainak megadásával, valamint egy sokkal könnyebb módon is, az email fiókján keresztül.

Abban az esetben, ha a felhasználó a hagyományos regisztrációs eljárást választja, és kitölti a kért adatokat (felhasználónév, email cím, ország, jelszó), a regisztrálása bekerül az adatbázisba, valamint egy alapértelmezett verifikációs eljárás megy keresztül. Ennek menete, hogy a regisztráció sikerességének pillanatában a felhasználóhoz egy *isVerified: false* mező társul az adatbázisban. Ennek szerepe és fontossága leginkább azon felhasználók kiküszöbölésére és feltartására szolgál, akik a webalkalmazást nem a megfelelő módon és nem a funkcionalitásaiért használná. A regisztráció teljesülése után a felhasználó által megadott email címre egy emailt küld a rendszer, melyben egy verifikációs link található. Erre kattintva meg tudja magát erősíteni a weboldalt használni kívánó személy. Amint ez megtörtént, az adatbázisban az *isVerified* mező értéke *true*-ra változik, és a felhasználónak elérhetővé válnak azon jogosultságok, melyekkel szerepkörét kihasználva minden, számára elérhető funkciót használatba tud venni.

Az alábbi kódrészlet a felhasználó regisztrálásának menetét mutatja az *auth.service.ts* fájlból:

```

register(username: string, email: string, password: string, country: string) {
  const body = { username, email, password, country };
  return this.http.post(`${environment.apiUrl}/api/register`, body).pipe(
    catchError(error => {
      console.error('Registration error:', error);
      return throwError(() => error);
    })
  );
}

```

TERVEZÉSI DOKUMENTÁCIÓ

Itt elindul egy POST a /api/register végpontra, ahol lefut a felhasználónév és az email cím ellenőrzése abból a célból, hogy nincs e valamelyik már regisztrálva az adatbázisban. Ha nincs, végrehajtódik a regisztrálás és bekerül a felhasználó az adatbázisba.

```
router.post("/register", validateUser, async (req: Request, res: Response) => {
  try {
    const { username, email, password, country } = req.body;

    // Felhasználó létezésének ellenőrzése
    const existingUser = await User.findOne({ $or: [{ username }, { email }] });
    if (existingUser) {
      return res.status(400).json({
        success: false,
        message: "Username or email already exists"
      });
    }

    const existingUserByEmail = await User.findOne({ email });
    if (existingUserByEmail) {
      return res.status(400).json({
        success: false,
        message: "Email already exists"
      });
    }

    // Jelszó hashelése
    const hashedPassword = await bcrypt.hash(password, 10);

    // Verifikációs token generálása
    const verificationToken = crypto.randomBytes(20).toString('hex');
    const verificationTokenExpires = new Date();
    verificationTokenExpires.setHours(verificationTokenExpires.getHours() + 24);

    const cfg = await getCredConfig();

    // Új felhasználó létrehozása
    const newUser = new User({
      username,
      email,
      password: hashedPassword,
      role: "user",
      date: new Date(),
      country,
      isVerified: false,
      credits: cfg.initialCreditsForNewUser ?? 0
    });

    await newUser.save();

    // Email küldése
    try {
      await issueVerification({
        _id: newUser._id,
        email: newUser.email,
        username: newUser.username,
      });
    } catch (emailErr) {
```

TERVEZÉSI DOKUMENTÁCIÓ

```
        console.error("[register] Verification email error:", emailErr);
    }

    return res.status(201).json({
        success: true,
        message: "Registration successful",
        userId: newUser._id,
    });
} catch (error) {
    console.error("Registration error:", error);
    return res.status(500).json({
        success: false,
        message: "Server error during registration",
    });
}
});
```

Amennyiben a felhasználó az email címével szeretne regisztrálni, a regisztrációs ablak adatainak kitöltését megkerülve és elfogadva a Google adatvédelmi irányelveit és általános szerződési feltételeit, egy lépésben elvégezheti a fiókkészítést.

Ennek folyamata a következő:

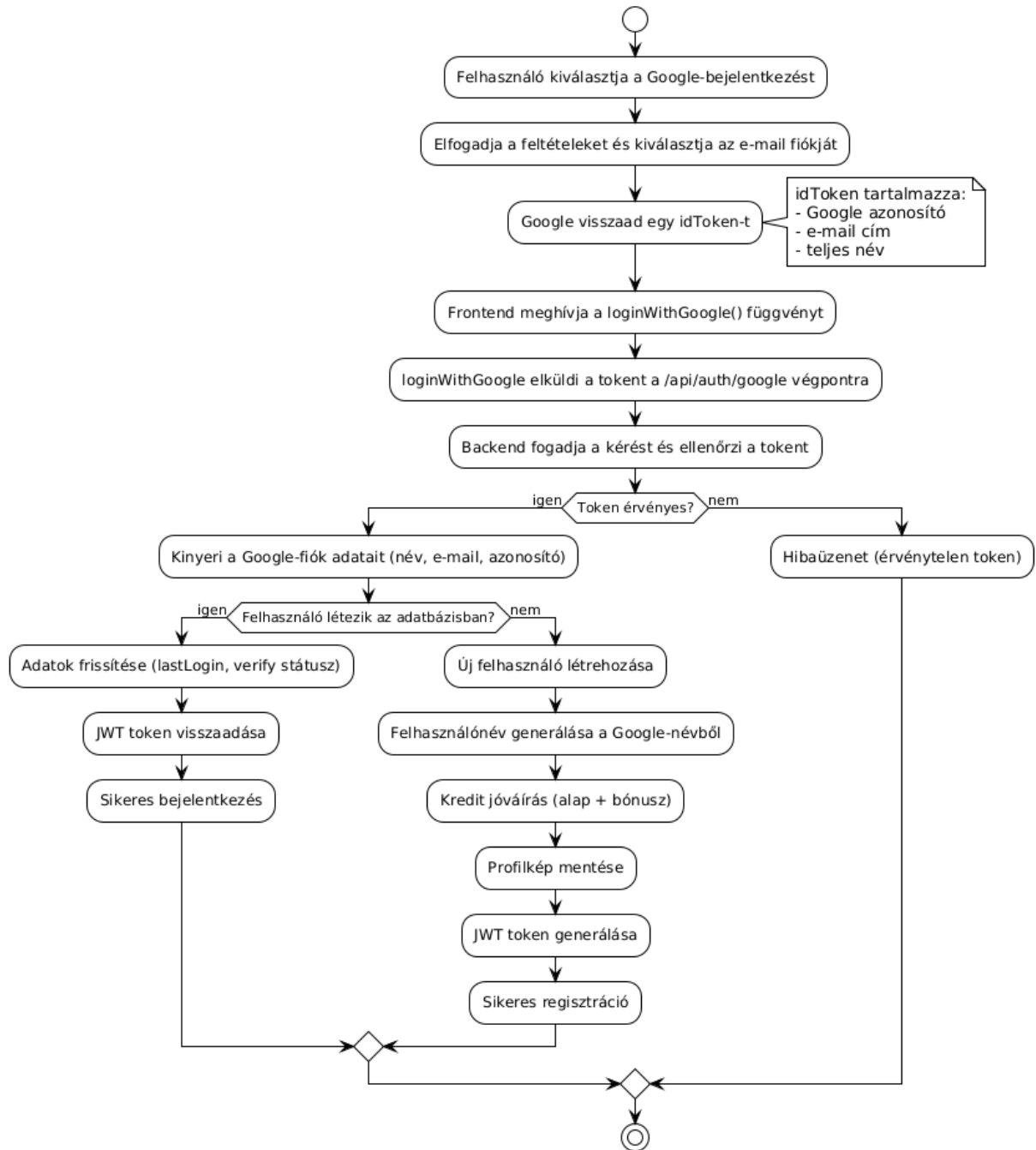
- Miután a felhasználó elfogadta a feltételeket és kiválasztotta az email fiókot, a Google egy úgynevezett idToken-t ad vissza, mely tartalmazza a felhasználó hitelesített adatait (pl. Google azonosító, email cím, teljes név)
- Meghívásra kerül a `loginWithGoogle` függvény, mely ezt a token-t továbbítja a `/api/auth/google` végpontra
- A szerver a `google-auth-library` csomag segítségével ellenőrzi a token érvényességét, valamint azt, hogy az a megfelelő Google Client ID számára lett-e kiállítva
- Amennyiben mindent megfelelőnek talált, a backend kinyeri a felhasználó gmail fiókjának adatait (email cím, név), majd ellenőrzi, hogy létezik-e már ilyen email cím vagy Google-ID az adatbázisban
- Amennyiben a felhasználó nem szerepel az adatbázisban, sikerrel zárul a regisztráció

Amennyiben a felhasználó a hagyományos regisztrációs menetet választja érvényes gmail címmel, abban az esetben a jövőbeli bejelentkezését elvégezheti az imént említett email-es bejelentkezési lehetőséggel. Ezzel elkerülhető az elfelejtett jelszó okozta bejelentkezési probléma.

A következő kódrészlet a `loginWithGoogle` függvényt tartalmazza:

```
loginWithGoogle(idToken: string) {
    return this.http.post<{ success: boolean; token: string }>(
        `${environment.apiUrl}/api/auth/google`,
        { idToken }
    ).pipe(
        tap(res => {
            if (res?.token) this.handleSuccessfulLogin(res.token);
        })
    );
}
```

TERVEZÉSI DOKUMENTÁCIÓ



13. ábra Google fiókkal való regisztrálás folyamatábrája

3.3.2. Jogosultságkezelés

A regisztrációt követő sikeres bejelentkezés után a felhasználó megkapja a *user* jogosultságot. Azonban a felhasználót, mint gyűjtőszót itt még két részre bontanám: megerősített és nem megerősített felhasználó.

Nem megerősített felhasználó

Azon felhasználók, akik nem a gmail címükkel regisztráltak az oldalra, automatikusan *nem megerősített* felhasználók lesznek. Ennek köszönhetően kezdetben számos olyan funkcióhoz nem férnek hozzá, mely a weboldal alapfunkcióinak tekinthető.

Ezek a user-ek a mások által feltöltött hangfájlokat le tudják játszani, azonban nincs hozzáférésük azokat letölteni, like-olni, kommentelni alá, valamint kedvencekbe helyezni. Mindezek mellett nem tudnak chatelni, jelenteni és loop-okat sem tudnak feltölteni, ezáltal krediteket szerezni. Ezt tulajdonképpen nevezhetjük „betekintőnek” az oldal funkcióit illetően, hisz megtapasztalja a weboldal kínálatát és lényegét, azonban a szolgáltatásait addig nem használhatja, míg meg nem erősíti az email címét.

Megerősített felhasználó

Azok a felhasználók, akik sikeresen átestek a verifikálás menetén, avagy Google fiókkal regisztráltak az oldalra, megkapják a *megerősített* státuszt. Ezzel minden olyan funkció elérhetővé válik, mely a nem megerősített user-ek számára nem: a feltöltött loop-ot nem csak lejátszani, de letölteni, like-olni és kedvencekbe helyezni is tudják, valamint megjegyzéseket írni alá. Mindezek mellett természetesen elérhető számukra a loop feltöltése opció, a kommentek, profilok, hozzászólások és a loop-ok jelentése, valamint a chatelés lehetősége.

Admin

Azok a felhasználók, akik admin jogosultsággal rendelkeznek, felelősek a weboldal tisztán tartásáért, a feltöltött adatok és fájlok moderálásáért és az oda nem illő tartalma szűréséért. Elérhető számukra az admin felület, ahová a jelentések bekerülnek – itt azokat elbírálják, és ha szükséges törlik a jelentett tartalmat. A felhasználókat kitilthatják, szükség szerint 1 napra, 1 hónapra vagy örökre, ezáltal megszűnik nekik azon jogkörük, melyeket a megerősített felhasználó szerepkör elérhetővé tett számukra. Elbírálják az új felhasználók első öt feltöltött loopjának sorsát – melyről a későbbiekben írok. Számukra azonban nem érhető el a jelentés funkció.

Kitiltott felhasználó

A kitiltott felhasználók csakúgy, mint a nem megerősített felhasználók, jogosultság megvonásban részesülnek. Számukra sem érhető el a loop letöltése és like-olása funkció, nem tudnak megjegyzéseket írni és jelenteni, valamint chatelni sem. Azonban tudnak a kedvencek közé hangfájlokat menteni. E funkció azért érhető el számukra, mert ha esetleg kitiltottként is a weboldalon járna és megtetszene neki egy hangfájl, azt ne kelljen újra megkeresnie amikor lejárt a kitiltása, hanem egyszerűen beteheti a kedvencei közé, hogy hamar megtalálja és letölthesse, amint ismét elérhető lesz számára e jogosultság.

3.3.3. Jogosultság ellenőrzés és validáció futásidőben

Az egyes jogosultságok ellenőrzéséhez elengedhetetlenek voltak az ezeket vizsgáló middleware-k létrehozása. A middleware-k olyan kódfájlok a backendben, melyek az ellenőrzésekhez, validálásokhoz és adott folyamatok szükséges függőségeinek ellenőrzésére szolgálnak.

Hitelesítés

A védett végpontok előtt első lépésben ellenőrizni kell, hogy van-e jogosultsága a felhasználónak elérni azt. Ehhez létrehoztam egy authenticateToken middleware-t, melynek működése az alábbi logikát követi:

- Megpróbálja kiolvasni a tokent a böngésző Authorization headerjéből
- Amennyiben sikerül, és megvan a felhasználó userId-je, emailje és roleja, ezen értékek továbbadónak a req.user-be
- Ha nincs token vagy nem sikerült kinyerni, a kérés 403-as hibakóddal leáll

Ennek köszönhetően kerül átengedésre azon felhasználó a végpont felé, melynek megfelelő jogosultságai vannak.

Az authenticateToken függvény az auth.middleware.ts fájlból:

```
export const authenticateToken = async (req: CustomRequest, res: Response, next:
NextFunction) => {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1];
  if (!token) return res.status(401).json({ message: "No token provided,
authorization denied" });
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET as string) as
jwt.JwtPayload;
    const freshUser = await User.findById(decoded.userId).select('role email');

    if (!freshUser) {
      return res.status(401).json({ message: "User no longer exists" });
    }
    req.user = {
      userId: decoded.userId,
      email: freshUser.email,
      role: freshUser.role
    };
    next();
  } catch (error) {
    console.error("[auth.middleware] Token invalid:", error);
    return res.status(403).json({ message: "Invalid or expired token" });
  }
};
```

Admin jogosultság ellenőrzés

Azon végpontok, melyek eléréséhez admin jogosultság szükséges, egy új middleware ellenőrzést kaptak. Ezen folyamat az előbb említett token kinyerés után válik relevánssá, hisz a már kinyert adatok alapján végez ellenőrzést. Amennyiben nem talál req.user-t, tehát a felhasználó nincs belépve, automatikusan megtagadja a végpont elérését. Ha talált, de a role nem admin, egy 403-as Belépés Tiltott üzenet várja a felhasználót.

Ezen middleware többek között olyan végpontokra van rátéve, mint a felhasználó kitiltása, kommentek vagy loop-ok törlése.

A requireAdmin függvény az admin.middleware.ts fájlból:

```
export const requireAdmin = (req: Request & { user?: any }, res: Response, next:
NextFunction) => {
  if (!req.user) {
    console.log("[requireAdmin] Error: No user in request");
    return res.status(401).json({ message: "Not authenticated" });
  }
  if (req.user.role !== "admin") {
    console.log("[requireAdmin] Error: User is not admin");
    return res.status(403).json({ message: "Forbidden - not admin" });
  }
  next();
};
```

Tiltás és verifikációs ellenőrzés

Mint azt már említettem, megkülönböztetünk nem megerősített és megerősített felhasználót, valamint tiltott felhasználót is.

A userAccess middleware olyan funkcióknál fut le, melyeket a megerősített és nem tiltott felhasználók érhetnek el. Ezek között van például a loop feltöltése és letöltése, a kommentelés, jelentés. Ennek folyamata a következőképp néz ki:

- Megnézi, hogy az adott felhasználó fiókja meg van-e erősítve
- Ha az adatbázisban a felhasználóhoz isVerified: false érték tartozik, akkor egy „EMAIL_NOT_VERIFIED” hibaüzenettel megtagadja a végpont elérését
- Amennyiben a felhasználóhoz tartozó bannedUntil mező dátuma a jövőben van, „BANNED” válasszal tagadja meg az elérést
- Ha a felhasználó megerősített fiókkal rendelkezik, és nincs aktív kitiltása, akkor továbbmehet

Ezen megoldás például azért szükséges egy ilyen weboldal megfelelő működéséhez, hogy a frissen regisztrált fiókkal, egy esetlegesen már korábban kitiltott felhasználó ne tudjon olyan interaktív funkciókat elérni, mint a hozzászólás vagy chatelés.

Példa ellenőrzés végpont elérésekhez:

/admin/reports végpont az adminok számára:

```
router.get('/admin/reports', authenticateToken, requireAdmin, listReports);
```

/loops/download/:id végpont a loop letöltésekhez:

```
router.get("/loops/download/:id", authenticateToken, checkVerifiedOrBanned,
requireDownloadCredit, downloadLoop);
```


Adatvalidáció

A jogosultságokon túl azt is szükséges ellenőrizni, hogy amit a felhasználó küld a backendnek, az formailag érvényes-e. Ezt külön validáló middleware-k végzik a `validation.middleware.ts` fájlban.

Többek között itt történik a regisztrálás során való ellenőrzés, mikor a felhasználónévnek, email címnek és jelszónak formailag meg kell egyeznie az elvártnak, valamint profiladatok frissítésénél megakadályozza a helytelen formátumú vagy túl hosszú adatok adatbázisba kerülését.

3.4. Loop-ok feltöltése, megjelenítése és hozzászólás

3.4.1. Loop-ok feltöltése

A hangfájlok feltöltése egy olyan folyamat kezdete, mely hosszútávon akár kollaborációkat, nagyszámú letöltés vagy akár egy kezdőleketet adhat a világhírű zenészek közé a feltörekvő producerek számára. Ahhoz, hogy mások megtalálják a saját zenéjükbe illő megfelelő loop-ot, szűréseket végezhetnek, melyeket megfelelően beállítva a weboldal kilistázza azon hangfájlokat, melyek a kategóriáknak megfelelnek, ezáltal beleillenek az elképzelt zenei alapba.

A hangsávok megfelelő tárolásának érdekében ezen adatokat a feltöltés során meg kell adnunk, melyeket később már csak az admin tud módosítani.

Ilyen adatok a következők:

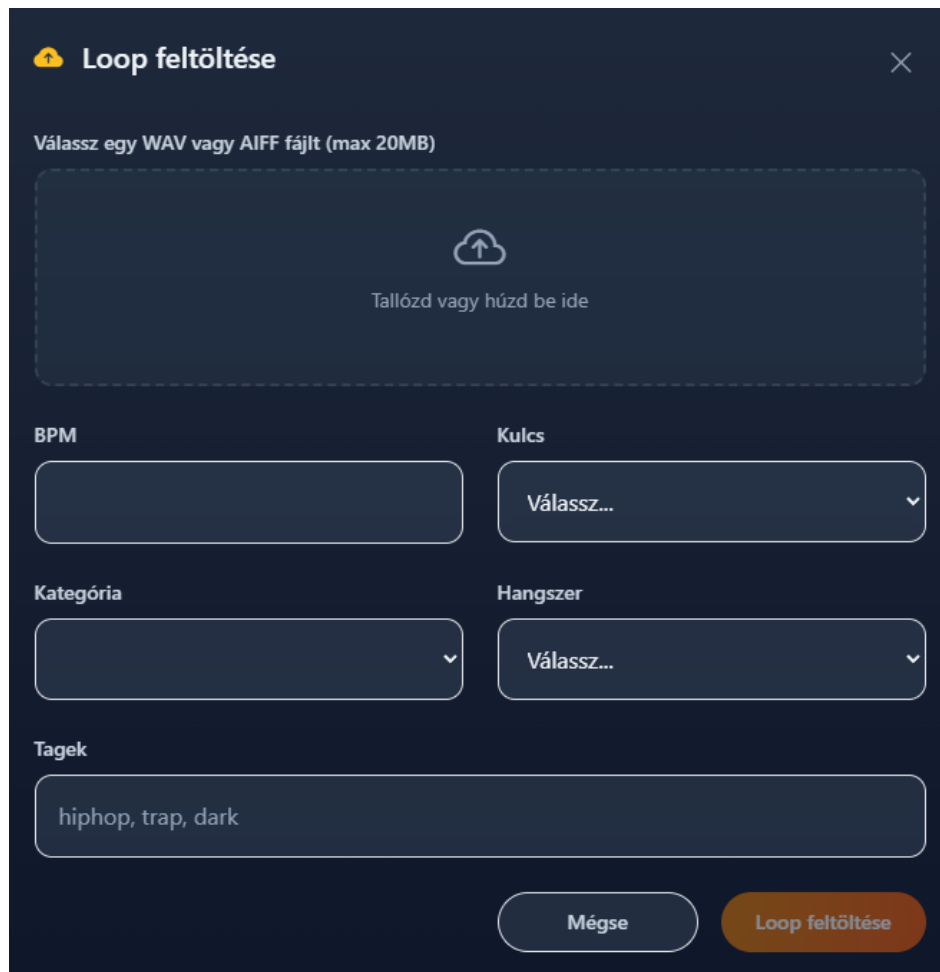
- BPM: egy szabványszám, mely pontosan meghatározza a loop tempóját, értéke minimum 30 és maximum 600 között lehet
- Kulcs: szabványérték, melyek pontosan meghatározza a loop hangnemét
- Hangszer: olyan értékeket tartalmaz, melyek pontosan meghatározzák a loop stílusát, a loop-ban található hangszereket (pl. zongora, hegedű, basszus, effekt, stb.)
- Kategória: olyan érték, mely pontosan meghatározza a hangfájl műfaját (pl. rap, hip-hop, rock, dance, stb.)
- Tagek: olyan szavak, melyekre egyértelmű szűréssel adott hangulatra lehet szűrni (pl. chill, hard, romantikus stb.)

Ahhoz, hogy egy egységes, minden zenekészítő program által elfogadott és veszteségmentes minőségű hangsávot tudjanak a felhasználók letölteni, a feltöltésnél csak .wav és .aiff formátumú fájl feltöltése engedélyezett. Míg a .wav kiterjesztésű fájlt minden zeneszerkesztő program feldolgoz, addig a .aiff fájlt inkább a macOS-en történő zenekészítés során szokták használni.

Alább fájlok kiterjesztését és a maximális méretét szabályzó kódrészlet látható a middleware-ből:

```
export const upload = multer({
  storage: multer.memoryStorage(),
  limits: {
    fileSize: 50 * 1024 * 1024, // 50MB
    fields: 10,
    parts: 20,
  },
});
```

```
fileFilter: (req, file, cb) => {
  const ok = [
    'audio/wav', 'audio/x-wav',
    'audio/aiff', 'audio/x-aiff'
  ].includes(file.mimetype);
  ok ? cb(null, true) : cb(new Error(Csak WAV/AIFF kiterjesztésű fájl tölthető
fel!
));
}
});
```



14. ábra. Loop feltöltése

Loop feltöltése új felhasználó esetén

Abban az esetben, ha egy olyan felhasználó szeretne loop-ot feltölteni, akinek korábban még nem volt feltöltése, új szabály lép életbe. A weboldalra feltöltött hangfájlok nem automatikusan válnak publikussá. Ennek okán egy feltöltött loop három kategóriába sorolható: pending (folyamatban), approved (jóváhagyott) és rejected (elutasítva). A Loop táblában található *status* mező hivatott ezt tárolni.

Ha a felhasználónak nincs legalább öt feltöltött hangfájlja, akkor az általa feltöltött loop automatikusan pending státuszba kerül. Ebben az esetben a tartalom nem jelenik meg a többi felhasználónak addig, míg ezt egy admin jóvá nem hagyja.

Ennek logikája a következő:

- Adott egy `approvedCount` változó, mely megszámolja, hogy a felhasználónak hány 'approved' státuszú feltöltése van
- Ha ez kevesebb, mint öt, az általa feltöltött loop automatikusan 'pending' státuszt kap, és a `/admin/loops` végpontra kerül elbírálásra
- Amint azt egy admin jóváhagyta, a feltöltött loop publikálásra kerül és a felhasználónak eggyel nő az 'approved' státuszú feltöltése az adatbázisban
- Amint megvan az öt feltöltött, 'approved' státuszú feltöltése, a következő feltöltések automatikusan publikálásra kerülnek

Emiatt a publikus loop lista nem az összes loop-ot adja vissza, csak azokat melyek 'approved' státuszúak.

Az alábbi kódrészlet ezt hivatott véghez vinni a `loop.controller.ts` fájlban:

```
const filter: any = { status: 'approved' };
const [items, total] = await Promise.all([
  Loop.find(filter)
    .populate("uploader", "username")
    .sort({ uploadDate: -1 })
    .skip(skip)
    .limit(limit)
    .lean(),
  Loop.countDocuments(filter),
]);
```

Loop-ok tárolása

A feltöltött loop-okat a fejlesztés korai szakaszában lokálisan, egy `uploads` mappában tároltam. Ennek hátulütője az, hogy a jövőben ha több felhasználó is használja az alkalmazást, elkerülhetetlen a hangfájlok valamilyen webtárhelyen történő mentése, hisz a fel- és letöltést tudnia kell kezelni a weboldalnak rengeteg kliens számára egyaránt. Ezért tárolásként az ingyenes, bár méretében korlátozott, a szakdolgozatom google fiókjához társított Google Drive szolgáltatást választottam. Ennek teljesítménye megfelelő, lokálisan futtatott backenden az egy oldalon található 1 hangfájlt 10 tesztetből 210.50 ms idő alatt kéri le és tölti be.

A következő kódrészlet ezt a teljesítménymérést mutatja be:

```
async measureAudioLoad(loopId: string) {
  // 1. URL összeállítás
  const audioUrl = this.getSafeAudioUrl(
    this.loops.find(l => l._id === loopId)?.path
  );
  if (!audioUrl) return;
  // 2. időmérés indítása
  const start = performance.now();
  // 3. hangfájl lekérése
  const response = await fetch(audioUrl);
  const audioData = await response.arrayBuffer();
  // 4. időmérés vége
  const elapsed = performance.now() - start;
  console.log(
    `[perf] Loop (${loopId}) betöltési ideje: ${elapsed.toFixed(2)} ms`
  );
}
```

```

);
const audioCtx = new AudioContext();
await audioCtx.decodeAudioData(audioData);
audioCtx.close();
}

```

3.4.2. Loop-ok megjelenítése

A feltöltött loop-okat a /loops végpont adja vissza. Ez egy listaszerű megjelenítés, melyben egymás alatt szerepelnek a Drive-ról lekért hangfájlok. Mindegyik fájlnál meg van jelenítve azon adatsokaság, mely alapján az behatárolható és szűrhető, valamint így a zenekészítő könnyedén megtalálhatja azt a fájlt, mely számára kedvező. A *user* jogkörrel rendelkezők számára a hangfájl címével egy sorban a BPM értéke, a like-olás, kedvencekhez adás, letöltés, valamint a jelentés lehetősége, mellette egy | jellel elválasztva pedig a like-ok száma látható. Az admin jogkörrel rendelkező felhasználók számára a jelentés ikon helyett egy törlés ikon van, valamint egy módosítási ablakot behozó *ceruza* ikon. A név alatt az adott loop kategóriája és kulcsa található, melyeket a tagek követnek. Alatta a lejátszó, melyre egy generált waveform is került.

A rendszer kétféle működési módot támogat, melyek közt a felhasználó egy gomb segítségével dinamikusan válthat, mely mentésre kerül. Ezen módok a következők:

- Teljesítmény mód (Gyors mód; alapértelmezett): Ebben a módban a kliensoldali erőforrások kímélése érdekében nem történik meg az összes fájl betöltése és elemzése. A waveform vizualizációja egy algoritmus segítségével generálódik, mely azonnali megjelenítést tesz lehetővé hálózati terhelés nélkül
- Minőségi mód (HQ Hullám): A rendszer valós időben elemzi a hangfájlokat, és tényleges amplitúdó-értékek alapján rajzolja ki a hullámformákat, mely az erősebb, illetve gyengébb, hangosabb és halkabb részeket hivatott jelölni, függőleges csíkok formájában.

Mindemellett a teljesítmény módban bevezettem egy interakció-alapú előtöltést. Ennek oka, hogy a fájlok azonnali lejátszása HTTP protokollon keresztül puffrelési késleltetést okoz. Ennek kiküszöbölése érdekében szükséges egy olyan előtöltés, mely még a lejátszás előtt elkezd betölteni a hangfájlt. Ezt úgy oldottam meg, hogy amint a felhasználó kurzora az adott loop kártyájába ér – és a loop korábban még nem került lejátszásra –, megindul annak betöltése. Míg a felhasználó a lejátszás gombhoz ér a kurzorra, addig eltelik 500-1000 millisecondum. Ez pont elég ahhoz, hogy az adott fájl első néhány másodperce már a memóriában legyen, így a lejátszás azonnal, vagy nagyon rövid időn belül elindul.

Alább a loop előtöltéséért felelős függvény található:

```

preloadAudio(loopId: string) {
  const audio = document.getElementById(`audio-${loopId}`) as HTMLAudioElement;
  if (audio && audio.preload !== 'auto') {
    audio.preload = 'auto';
    console.log(`[Preload] Letöltés indítása: ${loopId}`);
  }
}

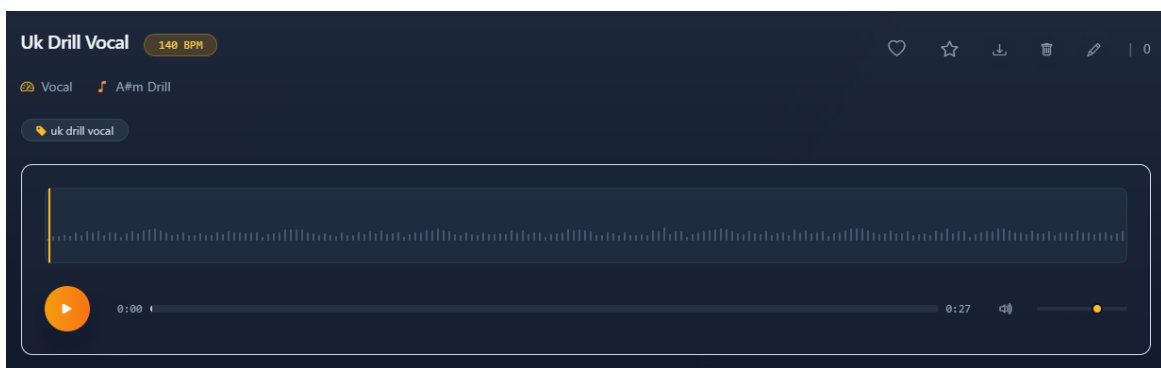
```

A HTML kódban ezt a függvényt egy mouseenter metódusra helyezve hívhatjuk meg:

```
(mouseenter)="preloadAudio(loop._id)
```

Kódrészlet a waveform megjelenítéséből:

```
if (this.useRealWaveforms) {  
  this.generateWaveform(loop._id);  
} else {  
  this.waveforms[loop._id] = Array(100).fill(0).map(() => Math.random() * 40 +  
10);  
}
```



15. ábra. Loop megjelenése

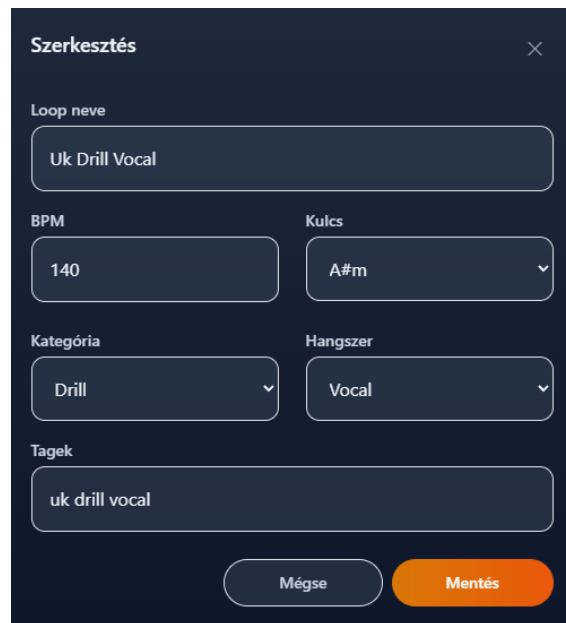
A waveform kiszámolásának menete:

- A loop betöltésével a komponens meghívja a waveformService.getOrCreate metódust
- A WaveformService lekéri a fájlt bináris formában
- Ez átadásra kerül a böngésző beépített Web Audio API-jának, mely visszaadja egy adott mintaátvételi pontnál az aktuális amplitudót
- A kód ezt a jelet 100 szeletre vágja, ami által minden a waveformban minden oszlop a hang egyenlő hosszúságú szeletét reprezentálja időben
- Minden szelet esetében összeadjuk az abszolút értéket, ezáltal megmérjük a jel erősségét az adott részen – nagyobb átlagos abszolút amplitudó esetében a zenerész hangosabb / ütősebb
- A visszaadott tömb bekerül a komponens waveforms[loopId] változójába
- Megjelenítésnél egy for ciklus végigmegy a tömbön, és egy csíkot rajzol ki a kiszámított érték alapján

Loop-ok szerkesztése

Az admin jogkörrel rendelkező felhasználók számára elérhető a loop-ok jelentése opció helyett a loop-ok szerkesztése lehetőség. Ezen gombot megnyomva felugrik a szerkesztő ablak, melyen az alábbi értékeket lehet módosítani:

- Loop neve
- Loop BPM száma
- Kulcs
- Kategória
- Hangszer
- Tag



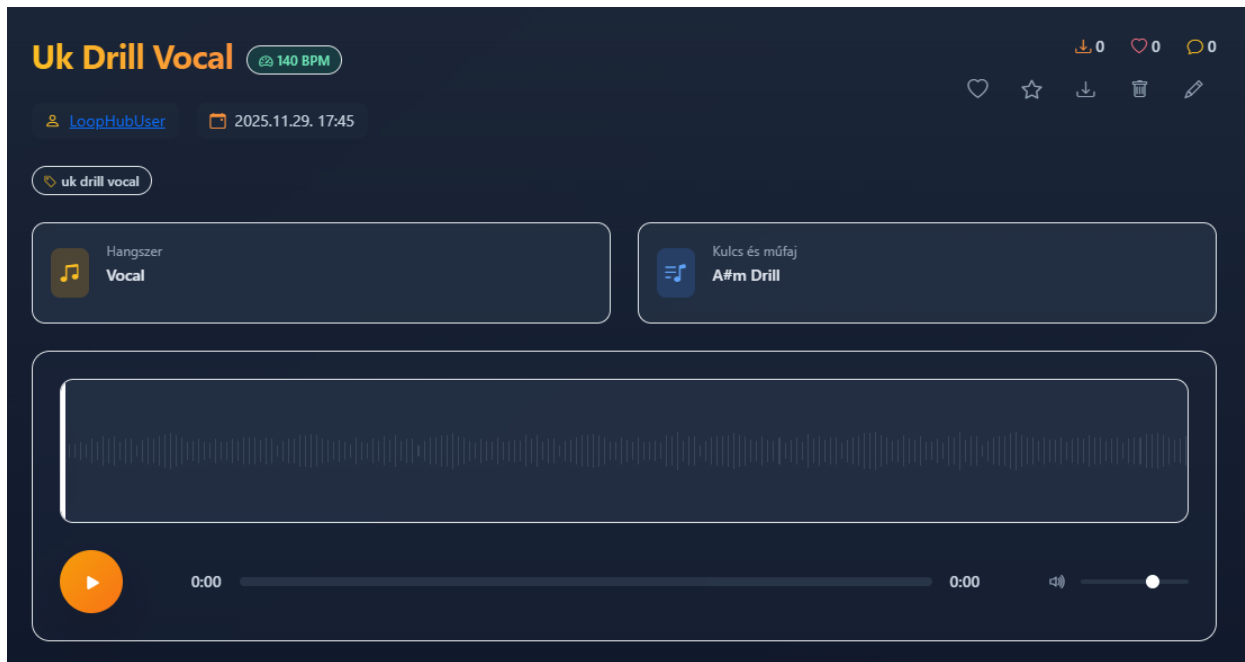
16. ábra. Loop-ok szerkesztése ablak

Részletes loop nézet

A loop-ra kattintva egy másik, úgynevezett *'loop-detail/id'* komponensre navigál a weboldal. Ezen nézet hasonlóan az előző loops listanézethez, a komponens elején a hangfájl adatait adja vissza, kicsit más elrendezésben. Annak okán, hogy minden loop-hoz tartozik egy részletes oldal, az elrendezésre való tekintettel itt a megjelenésnek már csak erről a fájlról kell szólnia.

Azon adatok, melyek a lista nézetben megtalálhatók voltak, itt is fellelhetők. Ezen felül tartalmazza a feltöltő nevét, melyre kattintva a profiljára navigálhatunk.

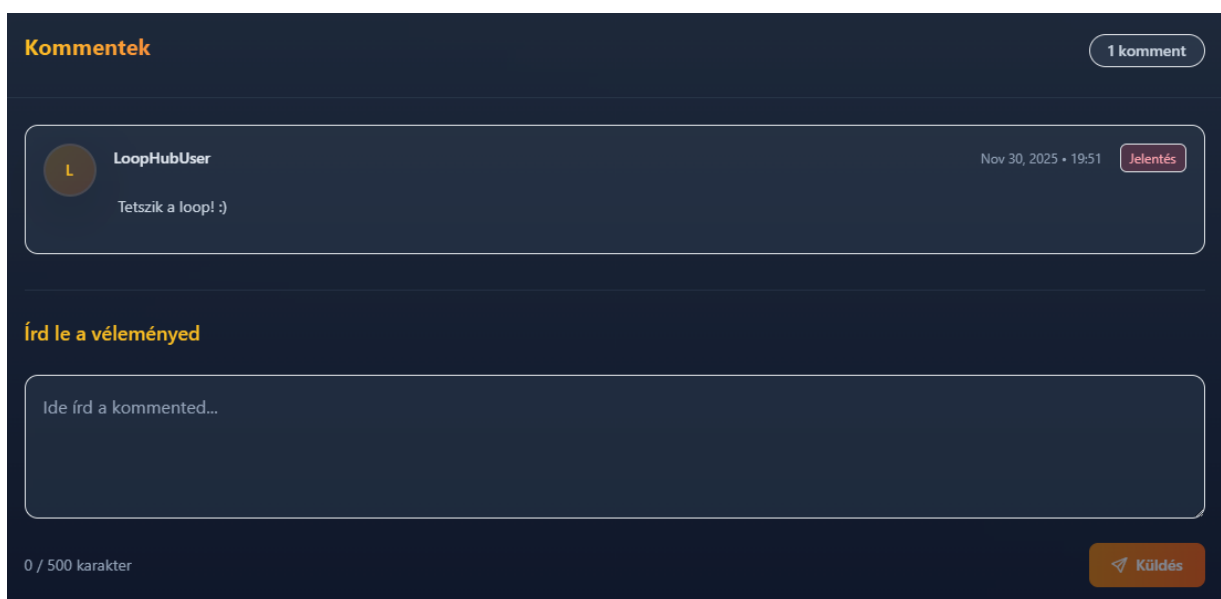
Mindemellett a feltöltés időpontja látható YYYY:MM:DD HH:MM formátumban, illetve a már korábban is leírt hangszer, illetve a kulcs és műfaj adathalmaz, egy nagyobb keretben és kiemelkedőbb designnal társítva.



17. ábra. Részletes loop nézet

3.4.3. Hozzászólás

Ezen ablak alatt megtalálható a feltöltött loop alá érkező kommentek sokasága. Egy komment maximum karakterszámát 500 karakterben határoztam meg. Egy komment létrehozásakor megkülönböztetünk admin és user jogkörrel rendelkező felhasználó által írt kommentet. Ahogy a loop-ok esetében, úgy itt is van lehetősége a user-nek jelenteni az adott kommentet, amennyiben sértőnek találja. Az adminnak ehelyett a már megszokott törlés lehetőség van jelen.



18. ábra. Hozzászólás egy loop alatt

Kódrészlet a komment hozzáadásáért felelős comment.controller.ts fájlból:

```
export const addComment = [
  authenticateToken,
  checkVerified,
  async (req: CustomRequest, res: Response) => {
    try {
      const { loopId } = req.params;
      const { text } = req.body;
      const userId = req.user?.userId;
      const commentingUser = await User.findById(userId);
      if (!commentingUser) {
        return res.status(404).json({ message: "Felhasználó nem található" });
      }
      const newComment = new Comment({
        text: text.trim(),
        user: userId,
        loop: loopId
      });
      const savedComment = await newComment.save();
      const populatedComment = await Comment.findById(savedComment._id)
        .populate('user', 'username profileImage role')
        .lean();
      res.status(201).json({
        success: true,
        data: populatedComment
      });
    } catch (error) {
      console.error("Hiba:", error);
      res.status(500).json({ message: "Szerver hiba" });
    }
  }
];
```

3.4.4. Like-olás

E funkció célja, hogy a felhasználó pozitív visszajelzést adjon egy másik felhasználó loop-jára. A megfelelő jogosultságellenőrzések után a like-ot szimbolizáló szív ikonra kattintva elindul egy POST folyamat a '/loops/:id/like' vagy '/loops/:id/unlike' végpontra. Az erre szolgáló kódrészlet:

```
router.post('/loops/:id/like', authenticateToken, checkVerified, likeLoop);
router.post('/loops/:id/unlike', authenticateToken, checkVerified, unlikeLoop);
```

A likeLoop és unLikeLoop folyamatok a következőképpen működnek:

- Betöltjük a loop-ot, ennek hiányában 404-es hibaüzenetet kapunk
- Ellenőrizzük, hogy a felhasználó like-olta-e már az adott loop-ot
- Frissítjük az állapotot: like-olás esetén a felhasználó belekerül a loop-hoz tartozó likedBy tömbbe, majd elmenti a változott értékeket; kilike-olás esetén pedig kiveszi a felhasználót a likedBy tömbből
- Like-olás esetén létrehozza a megfelelő értesítést az adott loop-ra vonatkozóan a feltöltő számára
- A frissült likeszám megjelenik a felhasználónak frontenden

Az unlikeLoop folyamat a loop.controller.ts fájlból:

```
export const unlikeLoop = [
  authenticateToken,
  checkVerified,
  async (req: CustomRequest, res: Response) => {
    try {
      const { id } = req.params;
      const userId = req.user.userId;

      const loop = await Loop.findById(id);
      if (!loop) {
        return res.status(404).json({ message: "Loop nem található" });
      }

      // a felhasználó like-olta-e
      const userIndex = loop.likedBy.findIndex(likedUserId =>
likedUserId.equals(userId));
      if (userIndex === -1) {
        return res.status(400).json({ message: "Még nem like-oltad ezt a loopot" });
      }

      // Like eltávolítása
      loop.likes -= 1;
      loop.likedBy.splice(userIndex, 1);
21

      await loop.save();

      res.json({ success: true, likes: loop.likes });
    } catch (error) {
      console.error("Unlike error:", error);
      res.status(500).json({ message: "Server error" });
    }
  }
];
```

A felhasználó számára történő gyors visszajelzéshez az úgynevezett „optimista frissítést” használtam, melynek lényege a következő: a loop like-olásánál vagy kilike-olásánál a gombra kattintva elindul a POST hívás, ezzel egyidőben automatikusan hozzáadódik vagy kivonódik az érték (jelen esetben az 1) a likeszámokból. Ezután a szerver visszaigazolja a megfelelő likeszámot, melyet frissít és felülír ha szükséges, hiba esetén pedig visszavonja a hozzáadott vagy kivont értéket.

Like-oláshoz kódpélda a loop-detail.component.ts fájlból:

```
isLikedByMe = false;
likes = 0;
loadingLike = false;

toggleLike() {
  if (this.loadingLike) return;
  this.loadingLike = true;
  const optimisticDelta = this.isLikedByMe ? -1 : +1;
  this.isLikedByMe = !this.isLikedByMe;
  this.likes = Math.max(0, this.likes + optimisticDelta);

  const req$ = this.isLikedByMe
```

```

? this.loopService.likeLoop(this.loop._id)
: this.loopService.unlikeLoop(this.loop._id);

req$.subscribe({
  next: (res) => {
    this.likes = res.likes;
    this.loadingLike = false;
    this.loopService.pushLikeUpdate({ loopId: this.loop._id, likes: res.likes,
likedByMe: this.isLikedByMe });
  },
  error: () => {
    this.isLikedByMe = !this.isLikedByMe;
    this.likes = Math.max(0, this.likes - optimisticDelta);
    this.loadingLike = false;
  }
});
}

```

3.4.5. Letöltés

A letöltési végpont az `api/loops /download/:id` vagy `api/loop-detail /download/:id` útvonalon érhető el. E végponthoz való hozzáférés csakúgy, mint a többi funkció esetében is, többszintű, először ellenőrizzük, hogy a felhasználó bevan-e jelentkezve, valamint, hogy nincs-e kitiltva és megvan-e erősítve a profil. Ezután az elegendő kreditszámot ellenőrizzük, mely a letöltés végbemeneteléhez szükséges. Az ezért felelős kódrészlet a `loop.routes.ts` fájlból:

```

router.get("/loops/download/:id", authenticateToken, checkVerifiedOrBanned,
requireDownloadCredit, downloadLoop);

```

A letöltés kiszolgálása a `downloadLoop` handlerben történik. Ennek lépései a következők:

- Loop letöltés gombot megnyomja a felhasználó
- Ellenőrizzük a szükséges jogosultságokat és a kreditszámot
- Ha minden megfelelő, növeljük eggyel a letöltések számát
- A megfelelő beállítás szerint kredit íródik jóvá a loop feltöltője számára
- A megfelelő elérhetőségről letöltődik a loop

Alább egy kódrészlet látható a letöltéskiszolgálásért és jutalmazásért felelős `downloadLoop` handlerből:

```

export const downloadLoop = [
  async (req: CustomRequest, res: Response) => {
    const { id } = req.params;
    const loop = await Loop.findById(id);
    if (!loop) return res.status(404).json({ success: false, message: "Loop not
found" });

    // 1) Letöltésszámláló
    await Loop.findByIdAndUpdate(id, { $inc: { downloads: 1 } });

    // 2) Feltöltő jutalmazása
    const cfg = await getCreditConfig();

```

TERVEZÉSI DOKUMENTÁCIÓ

```
const reward = Math.max(0, cfg.rewardPerDownloadToUploader || 0);
if (reward > 0 && loop.uploader) {
  await User.findByIdAndUpdate(loop.uploader, { $inc: { credits: reward }
}).exec();
}

// 3) Kiszolgálás
if (/^https?:\\\/\\\/i.test(loop.path)) {
  return res.redirect(loop.path);
}
const absolutePath = path.join(__dirname, '..', loop.path);
if (!fs.existsSync(absolutePath)) {
  return res.status(404).json({ success: false, message: "File not found on
server" });
}

res.setHeader('Content-Disposition', `attachment; filename="${loop.filename}"`);
res.setHeader('Content-Type', 'audio/wav');
return res.sendFile(absolutePath, (err) => {
  if (err && !res.headersSent) {
    res.status(500).json({ success: false, message: "Error sending file" });
  }
});
}
];
```

A letöltés gombra való kattintás után a felhasználó eszközére történő fájlmentés indításáért a loops.component.ts-ben loop-detail.component.ts-ben történő downloadLoop() függvény felel:

```
async downloadLoop(): Promise<void> {
  if (!this.loop?._id) return;
  try {
    const isVerified = await this.authService.isUserVerified().toPromise();
    this.loopService.downloadLoop(this.loop._id).subscribe({
      next: (blob: Blob) => { };
      const url = window.URL.createObjectURL(blob);
      const a = document.createElement('a');
      a.href = url;
      a.download = this.loop.filename || `loop_${this.loop._id}.wav`;
      document.body.appendChild(a);
      a.click();
      window.URL.revokeObjectURL(url);
      document.body.removeChild(a);
    },
    error: async (err) => {
      console.error('Download failed:', {
        error: err,
        loopId: this.loop?._id,
        path: this.loop?.path
      });
      if (err?.status === 401) { this.handleLogin(); return; }

      const payload = await this.readErrorResponse(err);
      if (payload?.code === 'BANNED') {
        alert(payload?.message || 'A fiókod tiltva van, a letöltés nem
```

```

engedélyezett.');
```

```

        return;
    }
    if (payload?.code === 'EMAIL_NOT_VERIFIED') {
        alert('You need to authenticate your email.');
```

```

        return;
    }
    alert(payload?.message || 'A letöltés nem sikerült.');
```

```

    }
});
} catch (e) {
    console.error('Error in download process:', e);
}
}

```

Az ehhez szükséges szolgáltatás a `loop.service.ts`-ben található. Ennek feladata egy GET kérés küldése a szervernek `responseType: 'blob'` beállítással, mellyel bináris adatokat kér a backendtől. E metódus felelős a `loop` fájl lekéréséért a backendről, és a fájlt bináris adatként visszaadja a komponensek. Ennek menete a következő:

- Kérést küld a szerverre
- Elküldi a bejelentkezett felhasználó tokenjét
- Bináris adatként (blob) fogadja a fájlt
- Átadja a `loop-detail.component.ts` fájlban található `downloadLoop()` függvénynek a letöltött adatot

Az erre vonatkozó kódrészlet a `loop.service.ts` fájlból:

```

downloadLoop(loopId: string): Observable<Blob> {
    return this.http.get(`${this.apiUrl}/api/loops/download/${loopId}`, {
        responseType: 'blob',
        headers: {
            'Authorization': `Bearer ${this.authService.getToken()}`
        },
        observe: 'response'
    }).pipe(
        tap(response => {
            console.log('Download response:', {
                status: response.status,
                headers: response.headers,
                body: response.body
            });
        }),
        map(response => response.body as Blob),
        catchError(error => {
            console.error('Download error:', {
                status: error.status,
                message: error.message,
                error: error.error
            });
            return throwError(() => error);
        })
    );
}

```

Ezt követően a kliensen megindul a letöltés.

3. táblázat. Letöltési folyamat rendszerszintű áttekintése.

Réteg	Fájl	Szerep	Mit csinál?
Backend	loop.controller.ts	Letöltés kiszolgálása	Ellenőrzi a jogokat, levonja/jóváírja a kreditet, és fájlt küld vissza a kliensnek
Service	loop.service.ts	Letöltés kérése	HTTP GET-et küld a backendnek, és választ bináris blokként fogadja
Komponens	loop-detail.component.ts	Letöltés elindítása a kliensen	A blobot menthető fájlá alakítja, és elindítja a letöltést

3.5. Kreditrendszer

Annak érdekében, hogy a weboldal önmagát is életben tudja tartani, egy dinamikus, hasonló webalkalmazásoknál nem fellelhető rendszert szerettem volna beépíteni. Valami olyat, mely kényszeríti és ösztönzi azon felhasználókat saját hangfájljuk feltöltésére, akik egyébként nem szerették volna loop-jaikat megosztani. Hisz ahhoz, hogy hangfájlokat tudjunk letölteni és válogatni közöttük, ahhoz egy bizonyos számú hangfájlnak elérhetőnek kell lennie. Minél több loop között keresgélhetnek a felhasználók, annál nagyobb élete lehet a projektnek. Ennek következtében hoztam létre és kötöttem hozzá a főbb funkciókhoz a kreditrendszert.

3.5.1. Működés

Mint azt már korábban említettem, egy felhasználó kreditpontokat tud szerezni és veszíteni tevékenységei folyamán. Ahhoz, hogy megmaradjon az egyensúly a le- és feltöltések aránya között, egy jól működő, egymással összefüggő rendszert alakítottam ki, melyek biztosítják a felhasználókat a hangfájlrendszer bővüléséről, és egyúttal elősegíti más loop-ok népszerűségét a letöltési számok növelésével.

3.5.2. Kreditszerzés és kreditvesztés

Ahhoz, hogy a user hangfájlokat tölthessen le a weboldallról, kreditet kell szereznie. Ehhez a felhasználónak nincs más dolga, mint feltölteni egy loop-ot. Ez a kreditrendszer alappillére, hisz az induló krediteken kívül csak úgy tudunk kreditet szerezni, hogy feltöltünk egy hangfájlt, vagy mások letöltik azt.

Ennek megfelelő működéséhez az alábbi változók létrehozása szükséges:

- initialCreditsForNewUser: új felhasználó ennyi kredittel indul
- bonusOnVerify: megerősítés után a felhasználónak ennyi kredit íródik jóvá
- creditsPerApprovedUpload: feltöltés után a felhasználónak ennyi kredit íródik jóvá

TERVEZÉSI DOKUMENTÁCIÓ

- **downloadCost:** letöltés után a felhasználó ennyi kreditet veszít
- **rewardPerDownloadToUploader:** a felhasználó ennyi kreditet kap, ha egy másik felhasználó letölti az egyik loop-ját
- **updatedAt:** az utolsó módosítás időpontja

Ezen értékeket az admin a weboldal admin panelén keresztül állíthatja.

Kódrészlet a kreditfrissítés menetéről az `admin.service.ts` fájlban:

```
export class AdminService {
  private base = `${environment.apiUrl}/api/admin`;
  constructor(private http: HttpClient) { }
  getCreditConfig() {
    return this.http.get<{ success: boolean; data: CreditConfigDto }>(`${this.base}/credit-config`);
  }
  updateCreditConfig(payload: Partial<CreditConfigDto>) {
    return this.http.patch<{ success: boolean; data: CreditConfigDto }>(`${this.base}/credit-config`, payload);
  }
}
```

Kreditek módosítása

Kezdő kreditek új felhasználónak

0

E-mail megerősítésért járó bónusz

2

Jóváhagyott feltöltésért járó kredit (feltöltőnek)

4

Letöltés költsége (levonás a letöltőtől)

2

Letöltés utáni jutalom (feltöltőnek)

1

Mentés

19. ábra. Kreditek módosítása

A kreditmentés folyamata:

- Amint az admin rányom a Mentés gombra, az új értékek elküldésre kerülnek a backendnek egy HTTP kérés request body-jában, mely így néz ki:

```
{
  "initialCreditsForNewUser": 1,
  "bonusOnVerify": 7,
  "creditsPerApprovedUpload": 9,
  "downloadCost": 5,
  "rewardPerDownloadToUploader": 3
}
```

- A creditConfig értékei frissülnek az új adatok szerint
- Majd a cfg.save() metódus elmenti az új értékeket az adatbázisba
- Meghívásra kerül az invalidateCreditConfigCache() függvény, mely törli a memóriában tárolt gyorsítótárat, így az értékek már az adatbázisból lekért adatok lesznek

Kódrészlet a creditConfig.ts fájlból:

```
export async function getCreditConfig(): Promise {
  const now = Date.now();
  if (_cache && now - _cacheTime < TTL) return _cache;
  let cfg = await CreditConfig.findOne();
  if (!cfg) {
    cfg = await CreditConfig.create({});
  }
  _cache = cfg;
  _cacheTime = now;
  return cfg;
}

export function invalidateCreditConfigCache() {
  _cache = null;
  _cacheTime = 0;
}
```

A letöltés előtt egy requireDownloadCredit nevű middleware fut le, mely lekéri az aktuális beállításokat a getCreditConfig() függvénnyel, majd ellenőrzi, hogy a felhasználó rendelkezik-e elegendő kredittel a letöltéshez. Ha nincs elég kredit, a rendszer 402-es hibakóddal elutasítja a letöltést, és jelzi a felhasználónak, hogy nincs elegendő kreditje. Ha van elegendő kredit, a letöltés után a megfelelő érték levonásra kerül a felhasználótól, növeli a downloadsTotal számlálót és elmenti a változott értékeket.

```
export async function requireDownloadCredit(req: CustomRequest, res: Response, next:
NextFunction) {
  try {
    const uid = req.user?.userId;
    if (!uid) return res.status(401).json({ success: false, message: 'Not
authenticated' });
    const cfg = await getCreditConfig();
    const cost = Math.max(0, cfg.downloadCost || 0);
    const user = await User.findById(uid).select('credits downloadsTotal');
```

```

    if (!user) return res.status(401).json({ success: false, message: 'Not
authenticated' });

    if ((user.credits ?? 0) < cost) {
        return res.status(402).json({
            success: false,
            code: 'NO_CREDITS',
            message: 'Nincs elég kredited a letöltéshez. Tölts fel loop-ot, hogy
creditet szerezhess!'
        });
    }
    if (cost > 0) user.credits = (user.credits ?? 0) - cost;
    user.downloadsTotal = (user.downloadsTotal ?? 0) + 1;
    await user.save();

    return next();
} catch (e) {
    console.error('[requireDownloadCredit] error:', e);
    return res.status(500).json({ success: false, message: 'Server error' });
}
}

```

3.6. Üzenetküldés

Szakdolgozatom egyik legfontosabb része a kapcsolatfelvétel. Míg más weboldalak esetében ez általában egy email küldés lehetőségben kimerül - jobb esetben, sokszor még ez sincs -, addig én egy könnyebb, egyszerűbb, felhasználóbarátibb megoldásban gondolkodtam. Mivel mindennapjainkat is körülöleli a folyamatos üzenetküldés, ezért ennek példájából fakadóan létrehoztam a chatelést, mint kapcsolatfelvételt.

3.6.1. Valós idejű chat modul

Az üzenetküldő komponens célja, hogy a felhasználók közvetlenül tudjanak csevegést indítani egymással, visszajelzést adni egy feltöltött hanganyagról egyszerre a feltöltőnek, vagy akár együttműködést kezdeményezni a platform elhagyása nélkül.

Hitelesítés és feltételek

A csevegést, mint funkciót csak bejelentkezett és megerősített felhasználó tudja használni, ezzel is nehezítve a nem kívánatos személyek sértő megjegyzéseinek küldését. Mindemellett tiltott felhasználó sem használhatja ezt a funkciót.

Kódrészlet az üzenetek lekéréséből és a hitelesítésről:

```

router.get('/messages', authenticateToken, checkVerified, async (req, res) => {
    const currentUserId = req.user.userId;
    const otherId = req.query.receiverId as string;
    if (!otherId) return res.status(400).json({ message: 'receiverId szükséges' });
    ...
});

```


Üzenetek lekérése

Az üzenetek lekéréséhez a /message végpont elérése szükséges. Ezen végpont adja vissza a teljes üzenetváltást két felhasználó között, megfelelően lejegyezve, időrendi sorrendben:

```
const messages = await ChatModel.find({
  $or: [
    { senderId: currentUserId, receiverId: otherId },
    { senderId: otherId, receiverId: currentUserId }
  ]
}).sort({ timestamp: 1 });
```

Ennek a lekérdezésnek a felépítése hasonlít a közismert megjelenítéshez, tehát jobb oldalon narancssárga színnel van jelölve a felhasználó által küldött üzenet, míg bal oldalt szürke kerettel annak a felhasználónak az üzenete, akivel épp a beszélgetést folytatjuk.

A végpontról a válasz nem csak a nyers üzeneteket tartalmazza, hanem a *users* objektumot is, ami a beszélgetésben résztvevő felhasználók publikus adatait tartalmazza:

- felhasználónév
- profilkép

```
const users = await User.find({ _id: { $in: ids } })
  .select('username profileImage')
  .lean();
```

A fejléc tartalmazza a chatpartner felhasználónevét és profilképét, valamint utolsó aktivitását.

Üzenetek tárolása

Mint ahogy arról az adatbázisoknál már írtam, az üzenetek ChatMessage séma alatt kerülnek tárolásra. Minden üzenet tartalmazza a következőket:

- senderId – aki az üzenetet küldi
- receiverId – a másik fél, akinek az üzenet küldésre kerül
- content – maga az üzenet
- timestamp – időpont, amikor az üzenet küldésre kerül

Minden rekord két felhasználó között jön létre, csoportos beszélgetésre nincs lehetőség.

Olvasottság jelölése

A felület a már leírtakon kívül mutatja azt is, hogy egy üzenet sikeresen kézbesítve lett, valamint azt, hogy a másik fél elolvasta-e az üzenetet. Erre szolgál a következő kódrészlet:

```
router.put('/messages/mark-read/:fromUserId', authenticateToken, checkVerified,
async (req: CustomRequest, res) => {
  const currentUserId = req.user.userId;
  const fromUserId = req.params.fromUserId;
  try {
    await ChatModel.updateMany(
      { senderId: fromUserId, receiverId: currentUserId, read: false },
      { $set: { read: true } }
    );
  }
```

TERVEZÉSI DOKUMENTÁCIÓ

```
res.json({ success: true });
} catch (error) {
  console.error('Hiba az üzenetek olvasottá állításakor:', error);
  res.status(500).json({ message: 'Szerver hiba' });
}
});
```

Ennek lényege, hogy mikor a címzett fél megnyitja a beszélgetést valakivel, minden felé érkezett üzenet olvasottá válik. A kliens oldalon ez a státusz az üzenetbuborék alatt jelenik meg. Alább az ezt tartalmazó kódrészlet található:

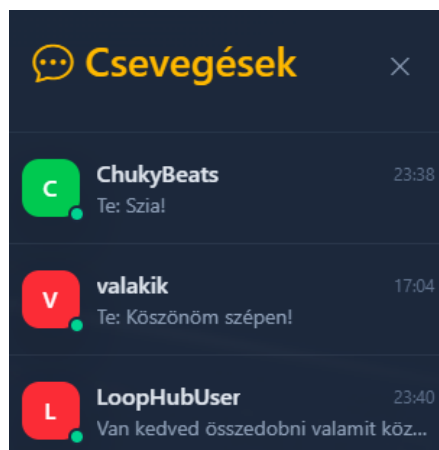
```
<div *ngIf="isLastOwnMessage(i)" class="flex items-center gap-2 text
xs text-slate-500 pl-12">
124

  <span>{{ message.timestamp | date:'HH:mm' }}</span>
  <span>•</span>
  <span>{{ message.read ? 'Olvasva' : 'Kézbesítve' }}</span>
</div>
```

A Socket.io réteg ezt tovább finomítja: amikor a másik fél megnyitja a chatet, a kliens automatikusan küldi a `markAsRead` eventet a szervernek. Ha a szerver visszajelez, a kliens végigmegy a helyi `messages` tömbön és beállítja `read = true` értékre.

```
this.socket.on('messagesReadByReceiver', ({ by }) => {
  if (by === this.receiverId) {
    this.messages.forEach(m => {
      if (m.senderId === this.currentUserId) m.read = true;
    });
  }
});
```

A navbaron megtalálható chat ikont lenyitva a beszélgetések listája tekinthető meg. Ez tulajdonképpen egy összefoglaló lista azon felhasználókkal, akikkel üzenetváltást folytattunk. Itt az utolsó küldött vagy fogadott üzenet jelenik meg.



20. ábra. Csevegések lista

Valós idejű üzenetküldés

Amikor a felhasználó megnyit egy beszélgetést, a kliens oldali komponens létrehoz egy Socket.io kapcsolatot a szerver felé.

Ez lokális fejlesztés alatt így néz ki:

```
this.socket = io('http://localhost:3000');  
this.socket.emit('joinRoom', this.senderId);
```

A kliens belép egy chatszobába, amely a felhasználó azonosítójával van azonosítva. Ennek célja, hogy csak a megfelelő címzett kapja meg az üzenetet.

Új üzenet küldése esetén a kliens küld egy Socket.io eseményt a szervernek:

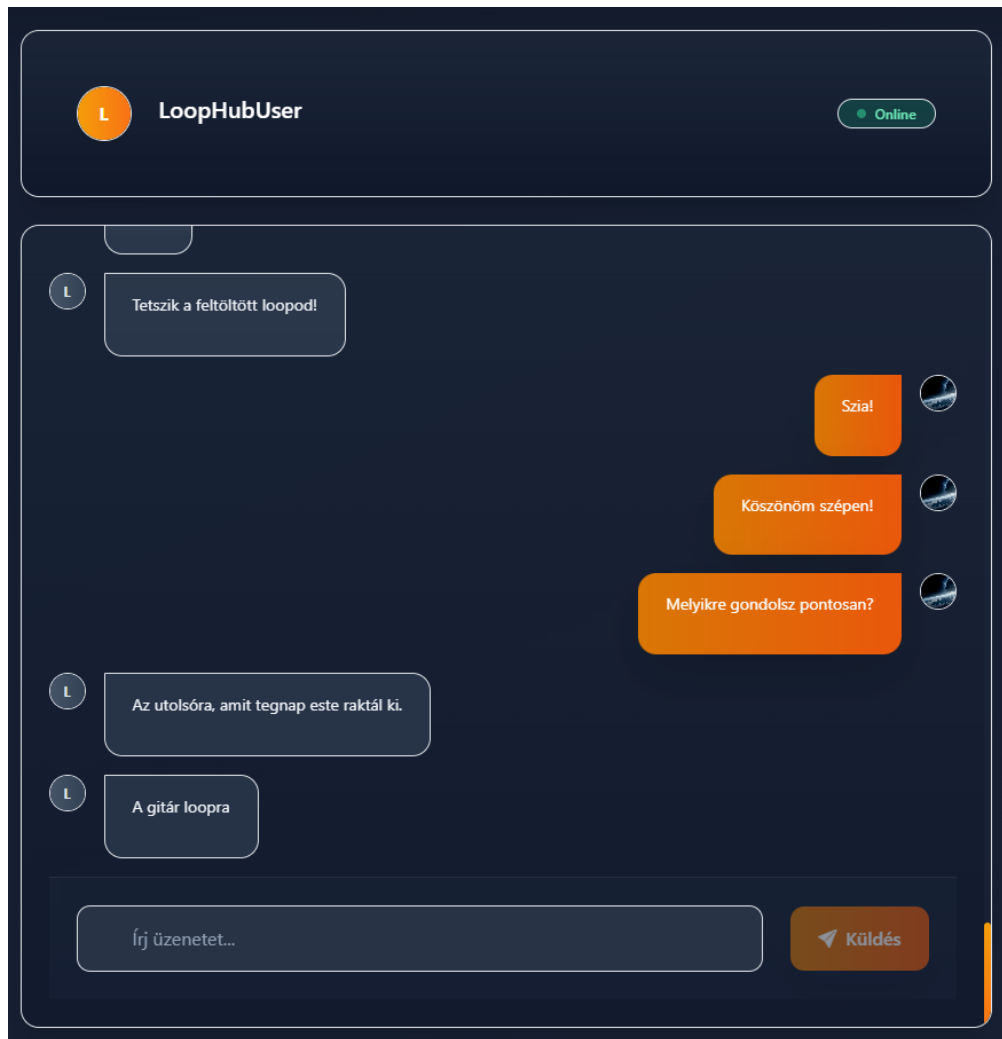
```
const message = {  
  senderId: this.senderId,  
  receiverId: this.receiverId,  
  content: this.messageContent.trim(),  
  timestamp: new Date()  
};  
  
this.socket.emit('sendMessage', message);
```

Ezen esemény tartalmazza a feladót, címzettet, tartalmat és az időpontot. Ez az adatbázisban eltárolódik, majd valós időben továbbítja azt.

Alább az ezért felelős kódrészlet látható a chat.component.ts fájlból:

```
this.socket.on('receiveMessage', (message: any) => {  
  if (message.senderId === this.receiverId || message.receiverId ===  
  this.receiverId) {  
    if (!message.timestamp) message.timestamp = new Date();  
    this.messages.push(message);  
    this.scrollToBottom();  
  
    this.socket.emit('markAsRead', { senderId: message.senderId, receiverId:  
message.receiverId });  
  }  
});
```

Ennek köszönhetően újratöltés nélkül, automatikusan megkapja a másik fél a küldött üzenetet, valamint a chatablak legörget a legújabb üzenetre és visszajelez, hogy megkapta azt.

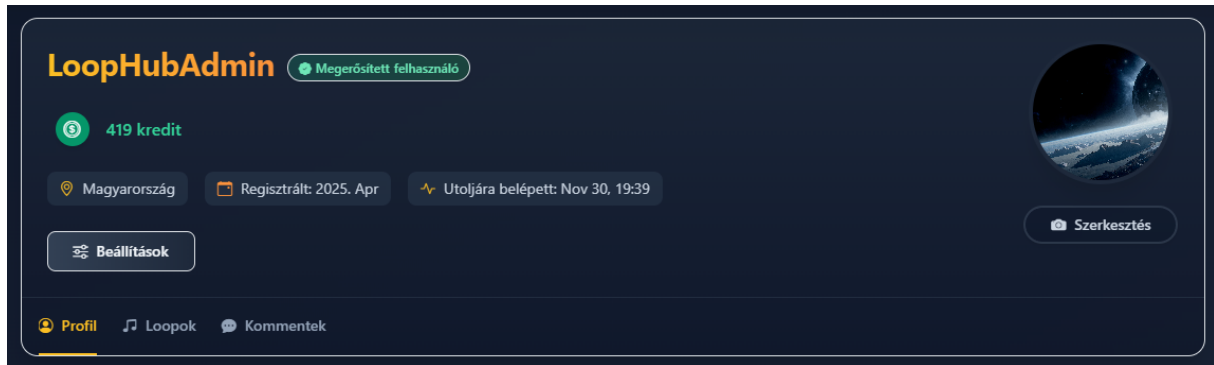


21. ábra. Chatablak megjelenítése

3.7. Profil oldal

A bejelentkezett felhasználó a /profile elérésével megtekintheti saját profilját. A komponens fejlécében a felhasználó nickneve, alatta a kreditszám, a bejelentkezéskor megadott ország, a csatlakozás dátuma és az utolsó bejelentkezés dátuma található. Vele egy sorban a profilkép található, mely szabadon módosítható bármilyen .png és .jpg kiterjesztésű képfájllal. Mindemellett megtalálható egy beállítások gomb, mellyel az alábbi adatok módosíthatók:

- Nicknév
- Email cím
- Jelszó
- Közösségi oldalak összekapcsolása a weboldallal



22. ábra. Profil fejléc

A fejléc alatt három menü található: Profil adatok, Loop-ok és Kommentek. A megjelenített adatok rácsszerű nézetben vannak, így jobban elkülöníthetők egymástól és mindegyik keretezett rész saját fontosságára hívja fel a figyelmet.

Profil adatok

A profil adatok alatt egyből a Rólam rész kerül megjelenítésre. Ez egy szöveges üzenet azon felhasználóknak, akik épp az adott profil oldalon tevékenykednek. Alatta a Közösségi oldalak rész található, ahová a felhasználó az alábbi oldalait csatolhatja:

- Facebook
- Instagram
- YouTube
- SoundCloud
- Spotify
- Tiktok
- X / Twitter
- Saját weboldal
- Email cím

Loop-ok

A második menüpont alatt az épp szemügyre vett felhasználó által feltöltött loop-ok találhatók. Csakúgy, mint korábban a loop listanézet esetében, itt is lehet szűrni a hangfájlok között, valamint megtekinteni a hozzájuk tartozó részletes loop oldalt. Itt nincs minden információ feltüntetve, csak a legfontosabbak, többek között az audiofájl neve, BPM értéke, hangneme, valamint a loop-ra érkező like-ok, kommentek, letöltések száma és a feltöltés időpontja. Ezen loop-ok a /api/loops végpontról vannak betöltve, az adott felhasználóra szűrve.

Alábbi kódrészlet a loop-ok felhasználóra szűrt lekéréséről a loop.controller.ts fájlból:

```
export async function listLoops(req: Request, res: Response) {
  try {
    const { uploader } = req.query;
    const filter: any = {};

    if (uploader) filter.uploader = uploader;
```

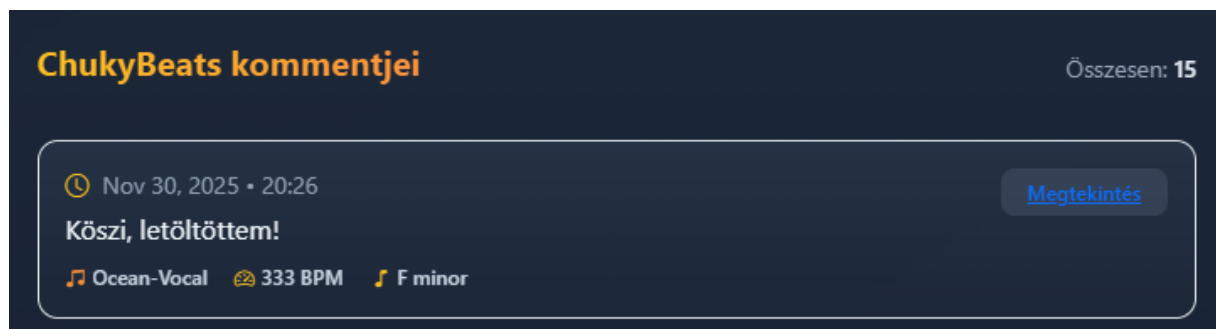
```
const loops = await Loop.find(filter)
  .sort({ createdAt: -1 })
  .select('_id filename bpm key scale duration downloads likes comments
uploadDate');

res.json({ success: true, items: loops, total: loops.length });
} catch (err) {
  console.error('Loop lista hiba:', err);
  res.status(500).json({ success: false, message: 'Server error' });
}
}
```

A loop-ok menüpont alatt mindemellett megtalálható egy Loop feltöltése gomb is, ami a loop-ok lista oldalára navigál, ahol elérhető a feltöltések lehetőség.

Kommentek

A fejléc alatt lévő harmadik menüpont a kommentek. Itt csakúgy, mint a felhasználó loop-jainak esetében, itt is szűrt kommentek olvashatók, melyeket az adott user tett közzé más loop-ok alatt. A listaszerűen megjelenített hozzászólások tartalmazzák azok keletkezési idejét, szövegének részletét, valamint az adott loop neve, BPM száma és kulcsa, ami alatt a komment született. Mellette egy megtekintés gomb található, mely az adott loop részletes oldalára vezet, ahol a hozzászólás megtekinthető.



23. ábra. Felhasználóra szűrt komment a profil oldalon

Egyéb lehetőségek a profil oldalon

Egy felhasználó profilján a már leírtakon kívül megtalálhatóak loop adatok, felhasználóhoz tartozó adatok és a műveletek menüpont.

Loop adatok alatt egy számszerű érték látható. Listázva van ugyanis a feltöltések és letöltések száma, valamint a loop-ok alá írt hozzászólások száma egy, az előzőekhez hasonló keretes elrendezésben. Az információk részen a felhasználónév, a beállított ország és az email cím található, míg a műveletek címszó esetében user jogosultsággal rendelkezők esetében az üzenetküldés és jelentés lehetőség, admin jogosultsággal rendelkezők esetében pedig az üzenetküldés és az egyéb adminisztratív lehetőségeket elérhetővé tévő Szankcionálás gomb.

3.8. Admin jogosultság

Az alábbiakban részletezem az admin jogkörrel járó lehetőségeket és kötelezettségeket, az elérhető funkciókat és velejáróit.

3.8.1. Admin menü

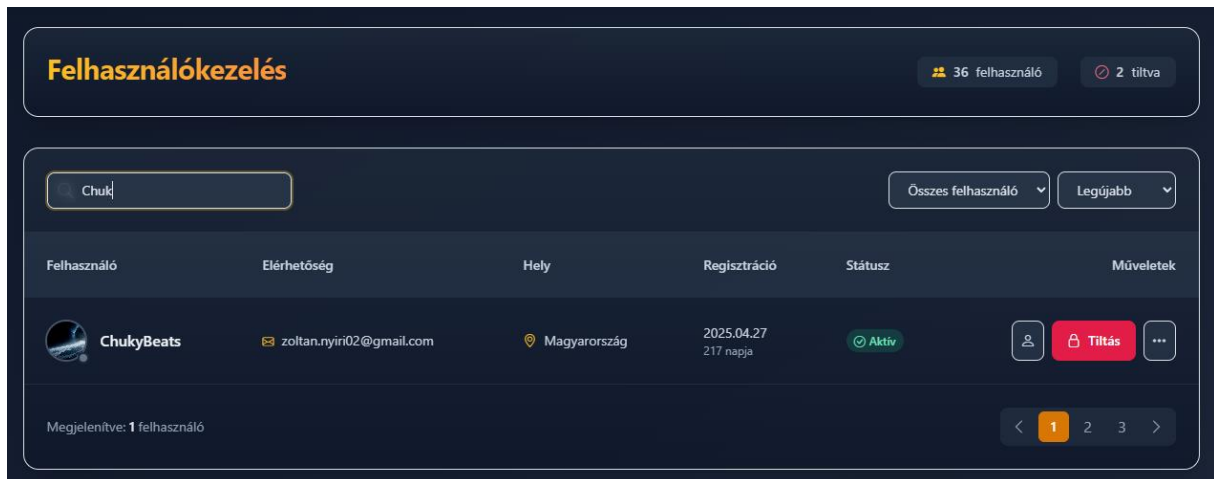
A /admin/dashboard oldalra lépve megtekinthető a csak adminok által elért admin felület összefoglaló adatokat tartalmazó komponense. Itt olyan, egész weboldalra lebontott adatok találhatók, mint a felhasználók, a jelenleg aktív felhasználók, a feltöltött loop-ok, kommentek, letöltések és like-ok száma, valamint egy grafikon a heti feltöltött loop-okról. Ezen ablakok közül a Felhasználók, Feltöltött loop-ok és Kommentek ablak az admin menü megfelelő oldalára vezet, melyek a felület sidebarján bal oldalt úgyszintén elérhetők.



24. ábra. Admin felület

A felhasználók menü megnyitása után listanézetben a weboldalra regisztrált felhasználók jelennek meg a következő adatokkal: felhasználónév, email cím, ország, regisztráció időpontja, státusza (aktív / bannolt-e vagy sem), valamint az akcióműveletek: profiljára vezető gomb, tiltás gomb – mely automatikusan 1 hónapra tiltja a felhasználót –, valamint egyéb műveletek gomb, mellyel a tiltás időtartamát állíthatjuk (1 napos, 1 hónapos, végleges), valamint admin jogosultságot adhatunk.

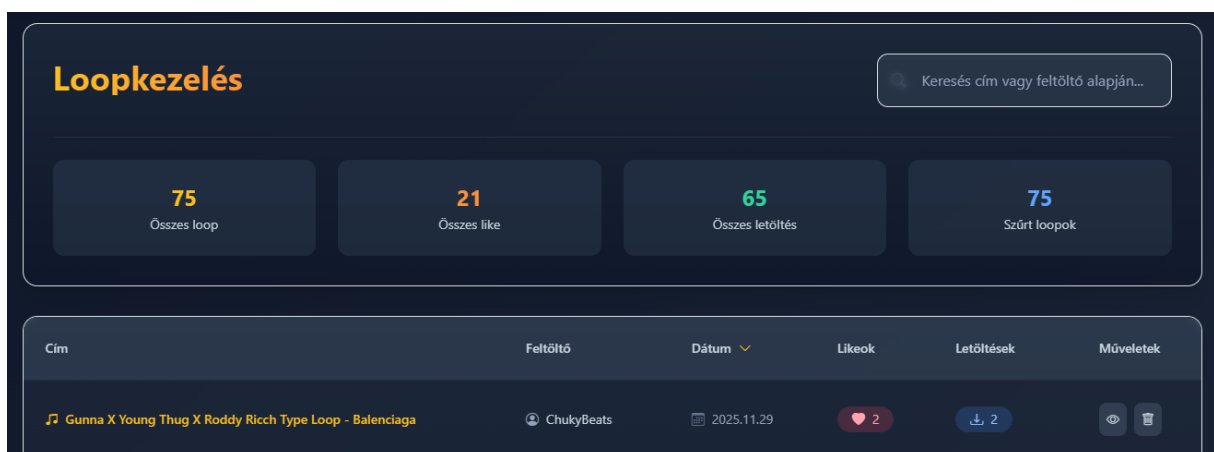
A lista fölött egy kereső mező található, mellyel felhasználó névre szűrhetünk, valamint a lista felsorolását szabályozó selectek, ahol szűrhetünk akár tiltott felhasználókra, aktív felhasználókra, valamint betölthetjük a listát legújabb vagy legrégebbi felhasználók szerint növekvő vagy csökkenő sorrendbe.



25. ábra. Admin menü - felhasználók lista

A loop-kezelés oldal megnyitásával egy – az előző példához hasonló – listaszerű megjelenítés során láthatók a weboldalra feltöltött loop-ok, a következő adatokkal ellátva: loop címe, feltöltője, feltöltésének dátuma, like-ok száma, letöltések száma és az akcióművelet: ezekre kattintva a részletes loop oldalra navigálhatunk, valamint törölhetjük a loop-ot. A loop címére kattintva egy ablak jelenik meg, ahol a loop lejátszható, valamint az alábbi részleteket tartalmazza a már említetteken kívül: BPM szám, hangnem és hangszer típusa.

A fejlécben itt is egy összefoglaló adatsokaság található: összegzi az összes loop-ot, like-ok számát, a letöltések számát és a publikus loop-ok számát, valamint egy keresőfelület is található, melyben a feltöltő neve vagy a feltöltött loop neve alapján tudunk szűrni a listában.



26. ábra. Admin menü – loop-ok listája

3.8.2. Jelentések létrehozása, kezelése és elbírálása

Mint ahogy arról már szó volt, a user jogosultsággal rendelkező felhasználók jelenteni tudják a mások által feltöltött loop-okat és kommenteket, valamint egy adott felhasználó profilját. Ezen jelentésablakok minden esetben ugyanazzal az elvárt input mezővel rendelkeznek: tehát a felugró ablak csupán egy szöveges üzenetet vár, melyben leírja a jelentő fél a problémáját az adott looppal / kommenttel / felhasználóval kapcsolatban.



27. ábra. Jelentés ablak

A jelentés küldése gombra kattintva a felhasználónak egy „Köszönjük! Jelentését megkaptuk” üzenet jelenik meg, majd az ablak automatikusan bezáródik. Ugyanez a folyamata a komment és a felhasználó jelentésének egyaránt.

A gomb lenyomása egy POST kérést küld az alábbi végpontok egyikére (típustól függően):

```
POST /api/reports/comments/:commentId
POST /api/reports/loops/:loopId
POST /api/reports/profiles/:userId
```

Az alábbi kódrészlet a loop jelentések létrehozásáért felelős a report.controller.ts fájlból:

```
export const createLoopReport = async (req: CustomRequest, res: Response) => {
  try {
    const { loopId } = req.params;
    const { message } = req.body;
    const reporterId = req.user?.userId;
    if (!reporterId) {
      return res.status(401).json({ success: false, message: 'Nem vagy
```

TERVEZÉSI DOKUMENTÁCIÓ

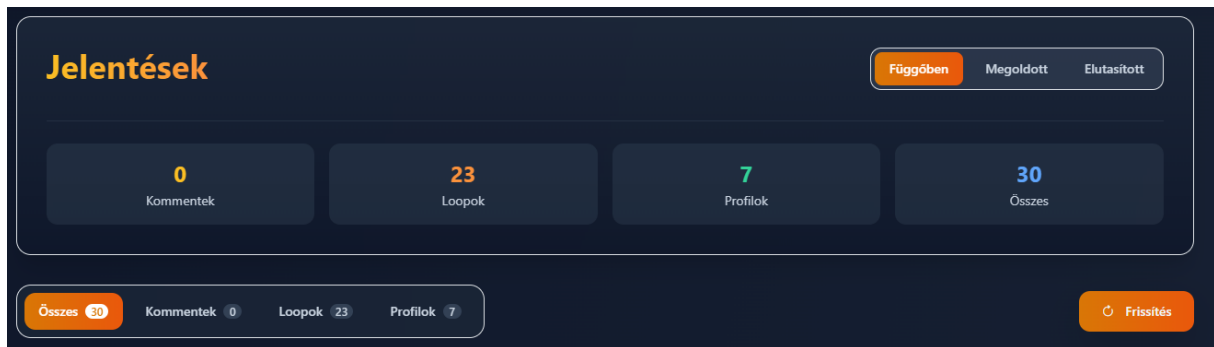
```
bejelentkezve.' });
}
if (!message || typeof message !== 'string' || message.trim().length < 3) {
  return res.status(400).json({ success: false, message: 'A jelentés indoklása
túl rövid.' });
}
const loop = await Loop.findById(loopId).populate('uploader', '_id username');
if (!loop) {
  return res.status(404).json({ success: false, message: 'A loop nem található.'
});
}
const doc = await Report.create({
  type: 'loop',
  targetId: loop._id,
  reporter: reporterId,
  reporterId: reporterId,
  targetOwnerId: loop.uploader?._id || loop.uploader,
  message: message.trim(),
  status: 'pending',
  meta: {
    loopId: loop._id,
    loopTitle: loop.filename
  }
});
return res.json({ success: true, data: doc });
} catch (err) {
  console.error('[createLoopReport] error:', err);
  return res.status(500).json({ success: false, message: 'Szerverhiba a jelentés
mentésekor.' });
}
};
```

A sikeres küldést követően a jelentés a Reports táblába kerül, a korábban már részletezett adatokkal.

Az admin felületen kialakított reports komponensre érkeznek be a felhasználók által küldött jelentések. Ezen felület a már részletezett *authenticateToken* és *requireAdmin* middleware-k által ellenőrzött végpontokról olvas. A következőkben a végpontokról látható kódrészlet, a *report.routes.ts* fájlból:

```
router.get('/admin/reports', authenticateToken, requireAdmin, listReports);
router.patch('/admin/reports/:id/status', authenticateToken, requireAdmin,
setReportStatus);
router.get('/admin/reports/counts', authenticateToken, requireAdmin,
getReportCounts);
```

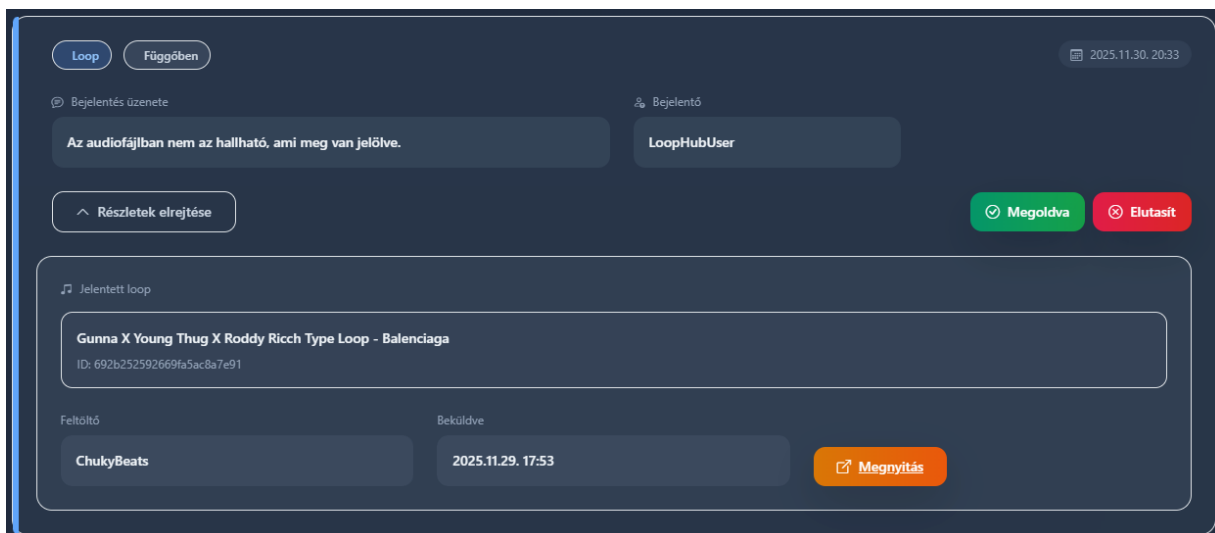
A bejelentések oldal az admin menü sidebarján látható. Megnyitva a */admin/reports* komponensen található, melyben egy részletesebb listában, egymástól kategóriáiban is elkülönítve jelennek meg a felhasználók által beküldött report-ok – külön menüpont alá érkeznek a kommentekre jött, a loop-okra jött, valamint a profilokra jött report-ok. Ezen felül van egy mindent összegző menüpont is, ami alá az összes report kilistázódik. A fejlécben itt is fellelhető egy rövid összefoglaló a jelentésekről. Minden beérkező report három kategóriába sorolandó: függőben, megoldott és elutasított. A függőben lévő a még el nem bírált, míg a másik két opció a megoldott vagyrepor elutasított jelentések halmazát tartalmazza.



28. ábra. Admin menü - jelentések

A megfelelő követhetőség érdekében a *report* résznél a loop-ra tett jelentést fogom itt is használni, hisz a logika és a működési elv minden jelentés tekintetében ugyanaz.

A beérkezett jelentés a listában tartalmazza a jelentés indokát, a bejelentő felhasználónevét és a keletkezés időpontját. A *Részletek megtekintése* gombra kattintva ezen adatok bővülne: megtekinthetjük a loop nevét és id-ját, a feltöltő neve és mellette még egyszer, jobban kiemelve a jelentés beküldésének dátuma. A megnyitás gombra kattintva átnavigálunk az adott loop-hoz tartozó részletes loop nézetre, ahol adott esetben törölhető a hangfájl vagy módosítható bármely érték. Elbírálás után a jelentések oldalra visszatérve elbírálható az adott report: kaphat megoldva vagy elutasítva státuszt. Ezek egy külön listába kerülnek, lényege egy esetleges korrigáció kívánta újraellenőrzés miatti könnyű kezelés, valamint a megfelelő kategorizálás. Ezek után a jelentés eltűnik a „függőben” listából.



29. ábra. Admin menü - jelentés elbírálása

3.8.3. Loop-ok módosítása

Mint ahogy arról már szó volt, egy feltöltött loop metaadatai (BPM, hangszer, kulcs, kategória, tagek és a loop neve) utólag is szerkeszthetők az adminisztrátor által. Erre akkor kerülhet sor, ha a feltöltő hibás adatot adott meg, vagy bármely indokból moderálásra szorul a tartalom.

A módosítás két helyről is elérhető:

- A loop-ok listáját tartalmazó loops oldalról
- Egy adott loop részletes adatait tartalmazó oldalról

A szerkesztést ábrázoló ceruza ikonra kattintva a felület megjeleníti a szerkesztő ablakot, amely előtölti az adott loop aktuális adatait. A módosítások itt végezhetők el, majd a Mentés gombra kattintva elküldi a backendnek.

A mentés során egy PATCH kérés kerül küldésre a `/api/admin/loops/:id` végpontra. A kérés tartalma a módosított mezőket tartalmazza JSON formátumban.

Kódrészlet a módosítás folyamatáról a `loop-detail.component.ts` fájlból:

```
export class LoopDetailComponent {
  loop: any = {};
  editForm: any = {};
  isEditModalOpen = false;
  editError = '';
  constructor(private http: HttpClient) {}
  // Mentés eseménykezelő
  saveChanges() {
    const payload: any = {};
    // csak a módosított mezők kerülnek küldésre
    if (this.editForm.filename) payload.filename = this.editForm.filename.trim();
    if (this.editForm.bpm) payload.bpm = Number(this.editForm.bpm);
    if (this.editForm.key) payload.key = this.editForm.key;
    if (this.editForm.scale) payload.scale = this.editForm.scale;
    if (this.editForm.instrument) payload.instrument = this.editForm.instrument;
    if (this.editForm.tags) {
      payload.tags = this.editForm.tags
        .split(',')
        .map((t: string) => t.trim())
        .filter((t: string) => !!t);
    }
    // PATCH kérés elküldése a backend felé
    this.http
      .patch(`${environment.apiUrl}/api/admin/loops/${this.loop._id}`, payload)
      .subscribe({
        next: (res: any) => {
          this.loop = res.data;
          this.isEditModalOpen = false;
        },
        error: (err) => {
          this.editError = err?.error?.message || 'A módosítás nem sikerült.';
        },
      });
  }
}
```

A backenden a beérkező kérés feldolgozása az `updateLoopAdmin()` függvényben történik.

A szerver a régi és az új adatokat összehasonlítja, majd a változott mezőket frissíti az adatbázisban.

Az adatbázis frissítésért a mongoose *Loop.findByIdAndUpdate()* metódusa a felelős:

```
const loop = await Loop.findByIdAndUpdate(id, update, { new: true });
```

Az ellenőrzések során a backend megnézi, hogy a frontend által küldött mezők megfelelnek-e a sémában definiált elvárásoknak. Sikeres mentés esetén a módosított loop adatok azonnal frissülnek az adatbázisban, majd az új értékekre frissülve módosulnak a hangfájl adatai.

3.8.4. Új felhasználó loopjának publikálása

Annak érdekében, hogy megbizonyosodjunk arról, hogy egy felhasználó valóban megfelelő hangfájlt tölt fel – és nem egy ide nem illőt –, az új feltöltők számára korlátozást vezettem be a feltöltést illetően: amennyiben a felhasználónak nincsenek még feltöltései, vagy nem elegendő feltöltése van, amíg el nem éri az öt jóváhagyott feltöltést, nem kerül automatikusan publikálásra a loop. Ez a következőt jelenti:

- A feltöltött loopról visszajelzést küld a backend, hogy admin általi jóváhagyásra vár
- A loop automatikusan bekerül a megfelelő státusszal az adatbázisba, ezek lehetnek: pending, approved, rejected
- A /admin/loop-moderation komponens ezeket lekérdezi és a szokásos módon, listaszerűen megjeleníti
- Az admin elbírálja: amennyiben a loop-ot alkalmasnak találja, a jóváhagyás gomb megnyomásával automatikusan megjeleni a /loops listában a loop. Az elutasítás gombra kattintva törlődik a loop. Ezen módosításokról a felhasználó értesítést kap

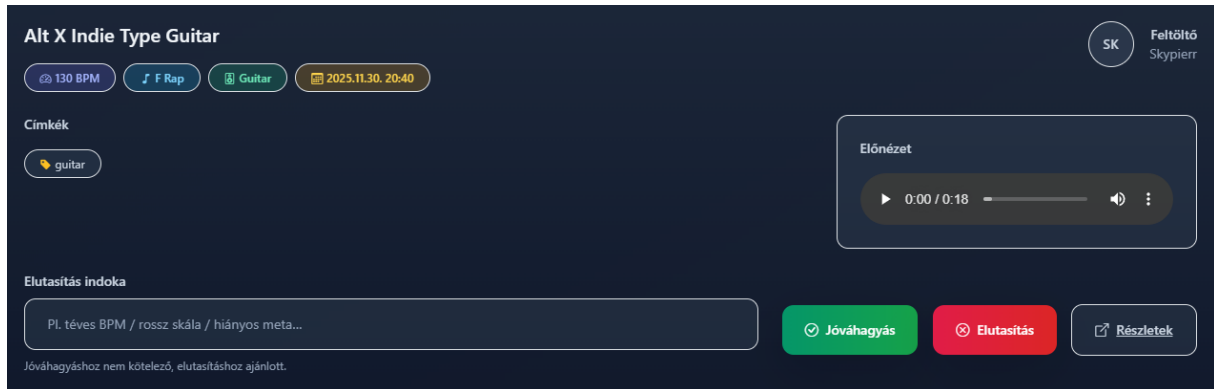
Az alábbi kódrészlet az új loop-okért felelős státuszokról és megfelelő kezelésükről:

```
// jóváhagyott feltöltések száma a feltöltőtől
const approvedCount = await Loop.countDocuments({ uploader, status: 'approved' });
const newLoop = new Loop({
  // ...
  status: approvedCount < 5 ? 'pending' : 'approved'
});
await newLoop.save();
// kreditek jóváírása automatikus publikálás esetén
const cfg = await getCredConfig();
if (newLoop.status === 'approved') {
  await User.findByIdAndUpdate(uploader, { $inc: { credits:
    cfg.creditsPerApprovedUpload ?? 0 } });
}
```

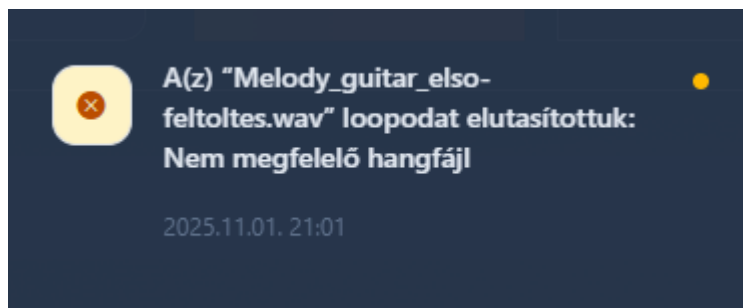
Amennyiben az admin jóváhagyta a loop-ot, lefut az approve() metódus, mely áthelyezi a loop-ot az approved státuszúakhoz, és megjelenik a /loops listában:

```
approve(loop: PendingLoop): void {
  if (!loop?._id) return;
  this.loopSvc.approveLoop(loop._id).subscribe({
```

```
next: () => {
  this.pendingLoops = this.pendingLoops.filter(l => l._id !== loop._id);
},
error: () => {
  alert('Jóváhagyás sikertelen.');
```



30. ábra. Admin menü - loop elbírálása



31. ábra. Példa értesítés az elbírálásról

3.9. Értesítések

A weboldalon több felhasználói és admin műveletek (pl. komment, like, moderáció) aszinkron módosításokat fejtenek ki a felhasználók által feltöltött tartalmakra. Annak érdekében, hogy a felhasználók átlátható, késleltetés nélküli visszajelzést kapjanak a saját tartalmaikkal kapcsolatos történésekről és módosulásokról, külön értesítési rendszert valósítottam meg.

3.9.1. Értesítések létrehozása

Az értesítések létrehozása a controllerekben történik. A letöltésekről, like-olásokról, admin által történő módosításokról vagy egyéb, a felhasználó általi feltöltésekben való változtatásokról a sikeres mentés után a felhasználó a navbaron található csengő gombra kattintva értesülhet. Ezen értesítések létrehozása a különböző módosítások sikeressége után meghívódó `Notification.create()` függvény segítségével történik.

Ennek menete a következő:

- A felhasználó vagy admin módosítást hajt végre a felhasználó által feltöltött fájlban / kommentben (like-olás esetén nő a likeszám, moderálás esetén változik egy érték, stb.)
- Amennyiben ezen művelet sikerrel befejeződött, a megfelelő mező frissül az adatbázisban
- Meghívódik a `notification.create()` függvény, mely az értesítések létrehozásáért felelős
- Ezen függvény beállítja a megfelelő értékeket (címezett felhasználó, a változtatást végbe vitt felhasználó, változtatott érték, változtatás típusa (like, komment, módosítás, stb.), üzenet a felhasználónak, valamint a fájl megnevezése (adott loop vagy komment)
- Az értesítés a megfelelő értékekkel mentésre kerül az adatbázisba
- Megjelenik a felhasználó számára az értesítések panelen

Mivel a metódus és az értesítések létrehozása minden esetben ugyanazon elven működik, a példában a like-olás után történő értesítéslétrehozást vezetem végig.

Amennyiben a like-olás művelete sikerrel végbement, és a megfelelő értékek mentésre kerültek az adatbázisban, lefut a `notification.create()` függvény, mely létrehozza az értesítést (feltéve, ha a like-oló nem a loop feltöltője is egyben).

Kódrészlet a `loop.controller.ts` fájlból:

```
if (!loop.uploader.equals(userId)) {
  const liker = await User.findById(userId);

  await Notification.create({
    userId: loop.uploader,      // Címzett: a loop feltöltője
    user: userId,              // A műveletet végrehajtó felhasználó
    type: 'like',              // Értesítés típusa
    message: `${liker.username} likeolta a loopodat: ${loop.filename}`,
    relatedItemId: loop._id     // A loop azonosítója
  });
}
```

A `notification.create()` mongoose metódus automatikusan menti az értesítést az adatbázisba, így nem szükséges a `.save()` hívás.

Miután az értesítés létrejött, a szerver visszaküldi az alábbi választ:

```
res.json({ success: true, likes: loop.likes });
```

A loop-ot feltöltő felhasználónál pedig megjelenik az értesítés a megfelelő formátumban.

Kódrészlet a megfelelő betöltésért felelős `notification.service.ts` fájlból:

```
this.notificationService.getNotifications().subscribe({
  next: (resp) => {
    this.notifications = resp.data;
    this.unreadCount = this.notifications.filter(n => !n.read).length;
  }
});
```

A felhasználó az értesítésre kattintva átnavigál a loop részletes oldalára. Ehhez a `handleNotificationClick()` függvény kerül meghívásra, amely a megfelelő, adott műveletet vagy változtatást megjelenítő komponensre navigál.

Kódrészlet a `handleNotificationClick()` függvényt tartalmazó `navbar.component.ts` fájlból:

```
handleNotificationClick(notification: any) {
  if (!notification.read) {
    this.notificationService.markAsRead(notification._id).subscribe({
      next: () => {
        notification.read = true;
        this.unreadNotificationsCount = Math.max(0, this.unreadNotificationsCount - 1);
      },
      error: (err) => console.error('Mark-as-read hiba:', err)
    });
  }
  const loopLikeTypes = ['comment', 'download', 'like', 'loop_approved', 'loop_rejected', 'loop_edited'];
  if (loopLikeTypes.includes(notification.type) && notification.relatedItemId) {
    this.router.navigate(['/loop-detail', notification.relatedItemId]).catch(err => {
      console.error('Navigációs hiba:', err);
    });
  } else {
    console.warn('Hiba történt a navigálás során');
  }
  this.showNotifications = false;
}
```

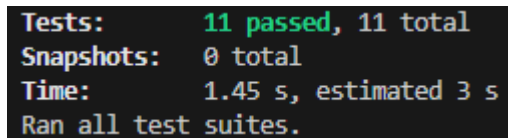
Amennyiben a felhasználó rákattint az értesítésre, annak státusza automatikusan olvasottra módosul és megtörténik a megfelelő végpontra történő navigálás.



32. ábra. Like értesítés

3.10. Tesztelés

A manuális tesztelés mellett az egyes tesztesetek létrehozásához a Jest.js [20] keretrendszert használtam. Ennek előnye, hogy megbízható és egyszerűen konfigurálható TypeScript alapú egységtesztek írására. A tesztesetek lefedik a főbb funkciókat, például a megfelelő adatbetöltést és hibakezelést, műveletkezeléseket és folyamatokat.



```
Tests:      11 passed, 11 total
Snapshots:  0 total
Time:       1.45 s, estimated 3 s
Ran all test suites.
```

33. ábra. Teszt eset.

3.11. A webalkalmazás továbbfejlesztése

Szakdolgozatomat a közeljövőben elérhetővé szeretném tenni, hogy éles környezetben segítse a produceri munkát. Ehhez azonban olyan funkciók továbbfejlesztése szükséges, melyek jövőbelátó problémákra is megoldással szolgálnak.

Fejlesztői szempontból szükséges egy olyan report rendszer kialakítása – amit a jelenlegi rendszer továbbfejlesztésével a legegyszerűbb elérni –, ahol a felhasználó a már funkcióiban működő lehetőségek mellett képes egyes üzeneteket is jelenteni. Ennek egyik módja lehet, hogy az admin jogkörrel rendelkezők is belelássanak az üzenetváltásba – mely jelenleg a szigorú biztonsági követelmények miatt csak a két, a beszélgetésben résztvevő fél számára érhető el. Továbbá célszerű egy olyan backend szolgáltatást választani, mely képes kezelni az egyidejűleg több kérést és műveletet futtató alkalmazást, és megfelelő időben kiszolgálni a felhasználókat. Végül, de nem utolsó sorban egy olyan felhő alapú tárhelyszolgáltatás bérlése is szükséges, melynek tárhelye nincs, vagy nagy méretben van csak limitálva – ezzel elkerülve az esetleges tárhely telítettséget.

4. FELHASZNÁLÓI DOKUMENTÁCIÓ

4.1. Rendszerkövetelmény

Szükséges és elegendő hardverkövetelmény:

- Legalább 512MB szabad grafikus memória
- Legalább 2048MB szabad rendszermemória
- Legalább 512MB szabad háttértár
- Windows XP vagy újabb operációs rendszer

4.2. Szoftverkövetelmény

- Angular 19.1.0
- Node.js 22.13.10 vagy újabb

4.3. Telepítés és elindítás

Szakdolgozatom fájljai tömörített formában elérhetők.

Név	Módosítás dátuma	Típus	Méret
.angular	2025. 08. 08. 20:21	Fájlmappa	
.vscode	2025. 08. 08. 20:13	Fájlmappa	
api	2025. 09. 27. 11:57	Fájlmappa	
node_modules	2025. 10. 14. 18:45	Fájlmappa	
public	2025. 08. 08. 20:13	Fájlmappa	
src	2025. 08. 08. 20:13	Fájlmappa	
test	2025. 10. 14. 19:09	Fájlmappa	
.editorconfig	2025. 08. 08. 20:13	Editor Config forr...	1 KB
.gitignore	2025. 09. 18. 17:16	Szöveges dokume...	1 KB
.postcssrc	2025. 08. 08. 20:13	JSON forrásfájl	1 KB
angular	2025. 09. 03. 17:48	JSON forrásfájl	4 KB
jest.config	2025. 10. 14. 19:08	JavaScript forrásfájl	2 KB
package	2025. 10. 14. 18:52	JSON forrásfájl	3 KB
package-lock	2025. 10. 14. 18:45	JSON forrásfájl	694 KB
proxy.conf	2025. 08. 08. 20:13	JSON forrásfájl	1 KB
README	2025. 08. 08. 20:13	Markdown forrásfájl	2 KB
setup-jest	2025. 10. 14. 18:47	TS fájl	2 KB
tsconfig.app	2025. 08. 08. 20:13	JSON forrásfájl	1 KB
tsconfig	2025. 08. 08. 20:13	JSON forrásfájl	1 KB
tsconfig.spec	2025. 10. 14. 18:53	JSON forrásfájl	1 KB

34. ábra. A szakdolgozat fájljai

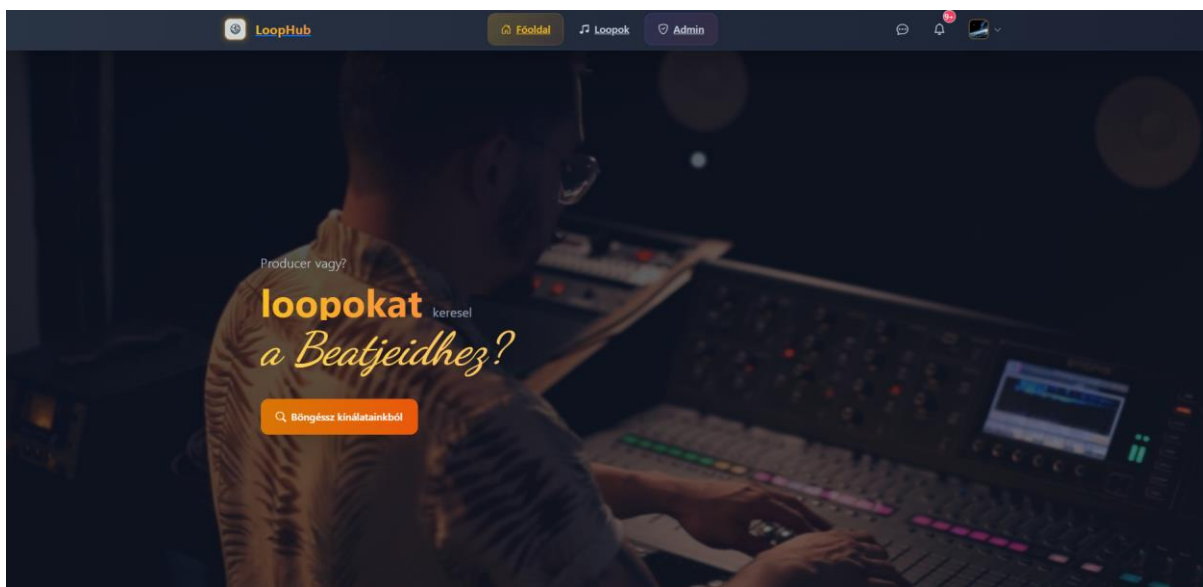
A függőségek telepítéséhez és az alkalmazás elindításához ezen műveletek végrehajtása szükséges:

- Terminál vagy egyéb, terminált tartalmazó alkalmazással meg kell nyitni a projektet
- A projekt gyökérkönyvtárába az `npm install` parancsot kell futtatni
- Az `ng serve -o` parancs kiadásával rövid időn belül automatikusan megnyílik a weboldal a `localhost:4200` alatt
- Az `npm run dev` parancs kiadásával rövid időn belül automatikusan elindul a backend szolgáltatás a 3000-es port alatt

4.4. Az alkalmazás használata

4.4.1. Kezdőoldal

A weboldalra lépve a kezdőképernyő fogad. Ezen komponens tartalmaz egy .mp4 kiterjesztésű fájlt, melyen egy producer látható. A képernyő közepén elhelyezve egy rövid összefoglaló olvasható. A Böngéssz kínálatainkból gomb a loop-ok listájára navigál.



35. ábra. Kezdőoldal

4.4.2. Regisztráció és bejelentkezés

A weboldal megfelelő használatához célszerű regisztrálni az oldalon, a kért adatok megadásával lehetséges.

Az alábbi adatok szükségesek a regisztráláshoz:

- Felhasználónév
- Email cím
- Ország választása a listából
- Jelszó
- Jelszó megerősítése

Amennyiben van már regisztrált fiók, úgy a Jelentkezz be gombra kattintva átnavigálunk a bejelentkezés földre, ahol a megfelelő adatok megadásával elvégezhető a bejelentkezés. A regisztráció lépése megkerülhető, amennyiben a felhasználó a Google fiókjával lép be a weboldalra. Ebben az esetben, miután elfogadja a Google adatvédelmi szerződését, sikeres bejelentkezést tudhat maga mögött.

36. ábra. Regisztrációs ablak

37. ábra. Bejelentkezés ablak

Egyes funkciók eléréséhez (például le- és feltöltés, kommentelés stb.) kötelező az email cím megerősítése. Amennyiben Google fiókkal történt a belépés, ez automatikusan megtörténik. Regisztráció esetén a regisztrált email címre a rendszer automatikusan küld egy emailt, ahol elvégezhető a megerősítés

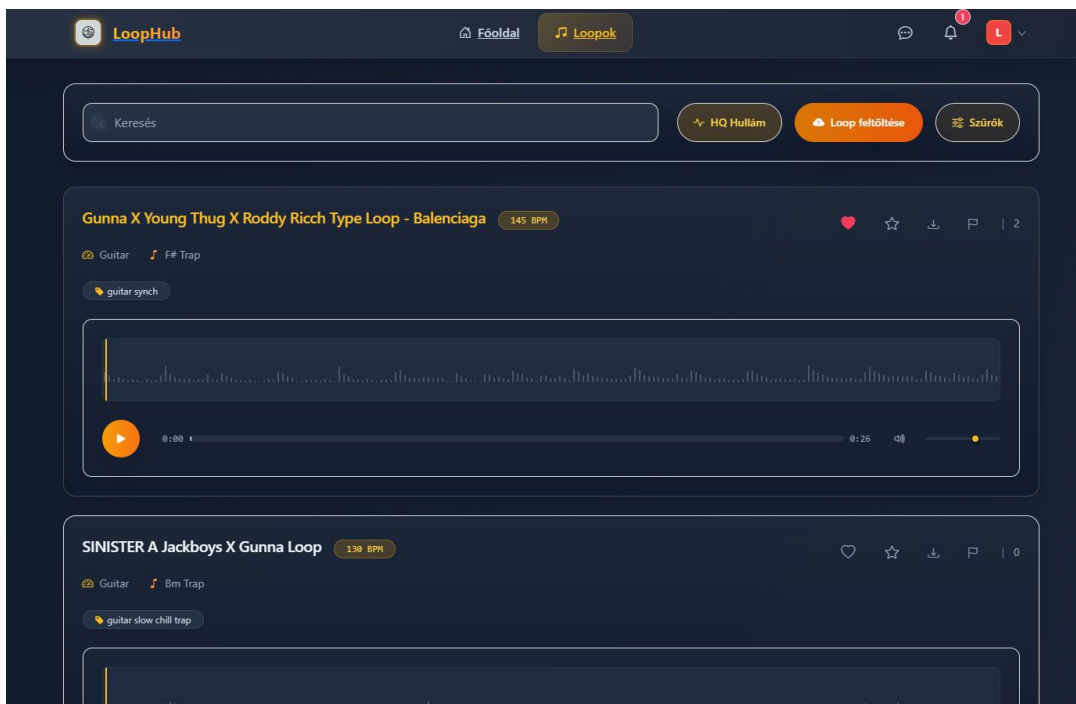
4.4.3. Kreditrendszer

A weboldalon kreditrendszer került bevezetésre. A feltöltések és a loop-jaink másol által történő letöltése által kreditek szerezhetők, a letöltések pedig kreditekbe kerülnek. Az alapértelmezett kreditmozgások az alábbiak:

- Szerezhető kreditek: email megerősítésért (2), loop feltöltésért (4), mások által történő letöltésért (1)
- Veszíthető kreditek: letöltésért (2)

4.4.4. Loop-ok

A loop-ok listaoldalára lépve megjelennek az elérhető loop-ok wav vagy aiff formátumban. A keresőben, valamint a szűrőket alkalmazva lehet szűrni a feltöltött loop-ok között, hogy olyan fájlok közül válogathassunk, melyek számunkra megfelelőek. Mindemellett itt tölthető fel loop, valamint elérhetők azon akciógombok, mint like-olás, kedvencekbe helyezés, letöltés és jelentés.



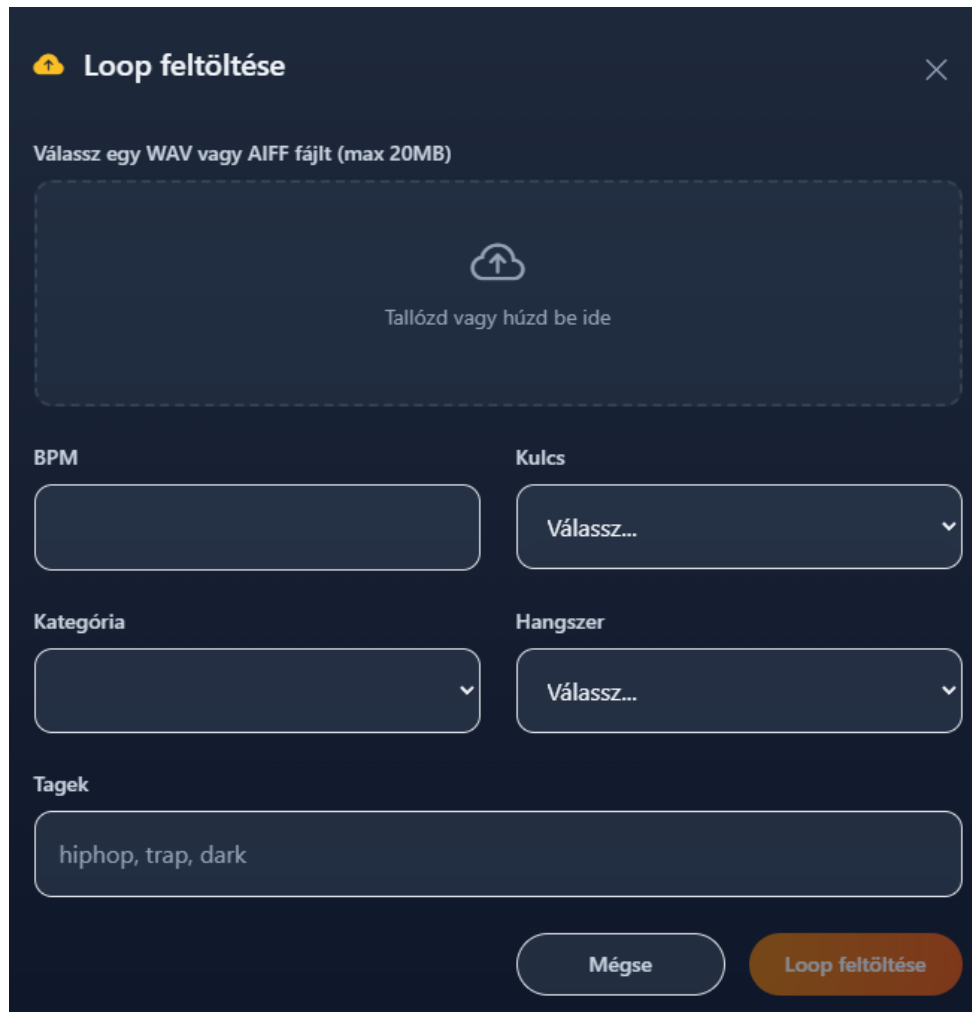
38. ábra. Loop-ok listája

Kedvencekbe helyezett loop-ok

Annak érdekében, hogy kedvenc loop-jaink hamar megtalálhatók legyenek, a profil menüjében kiválasztható a kedvencek menüpont, mely egy, az általunk kedvencekbe helyezett loop-okkal teli oldalból áll, melynek funkcióiban megegyeznek a loop-ok listaoldallal.

Loop-ok feltöltése

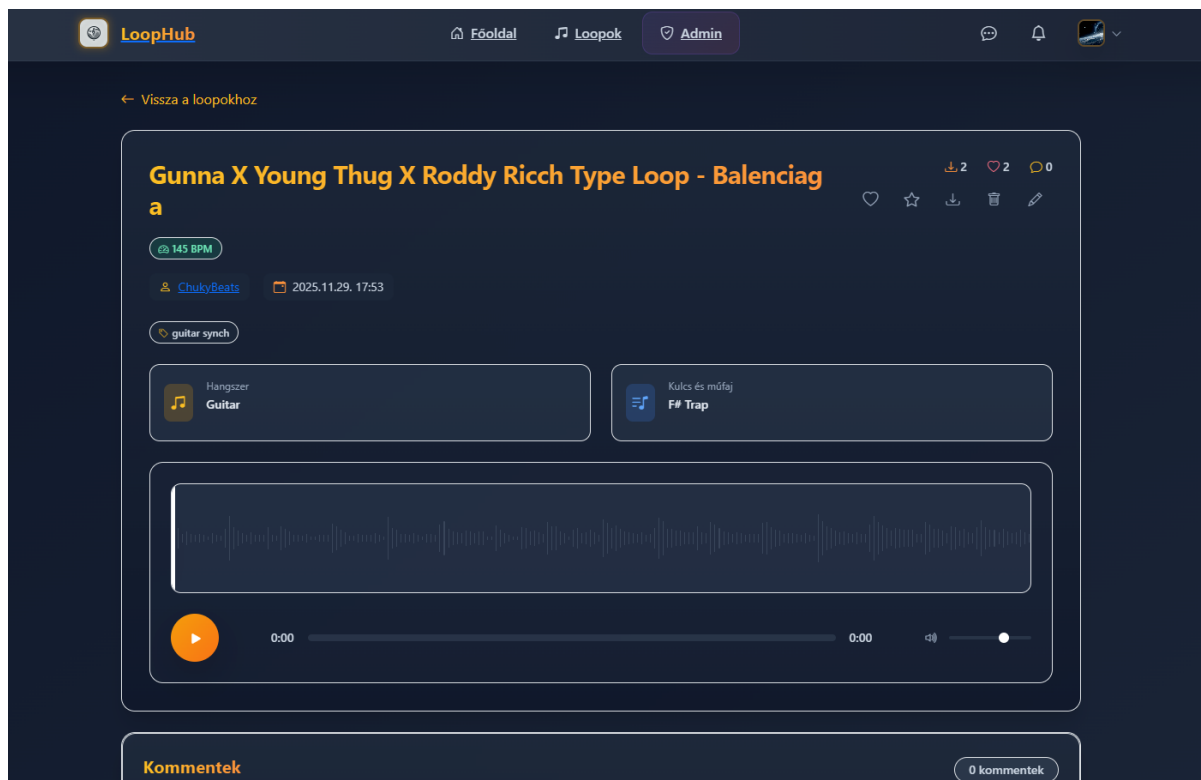
A loop-ok feltöltése a loop-ok oldalon lehetséges a megfelelő értékek megadásával. Csak .wav, illetve .aiff kiterjesztésű fájl tölthető fel, valamint a maximális méret 20MB.



39. ábra. Loop feltöltés

Minden felhasználó első 5 feltöltése automatikusan bekerül egy ellenőrzésre. Amint elfogadásra kerülnek, a felhasználó értesül róla a felső sávban található harang által szimbolizált értesítési panelen keresztül. Az 5 jóváhagyott feltöltés után a többi automatikusan megtörténik.

Egy loop-ra kattintva megjelenik annak részletes oldala. Mindazon funkciók, melyek a loop listák esetében elvégezhetők voltak itt is elvégezhetők. Mindezen felül részletesebb leírás található a loop-ról, valamint hozzászólási lehetőséget is tartalmaz.



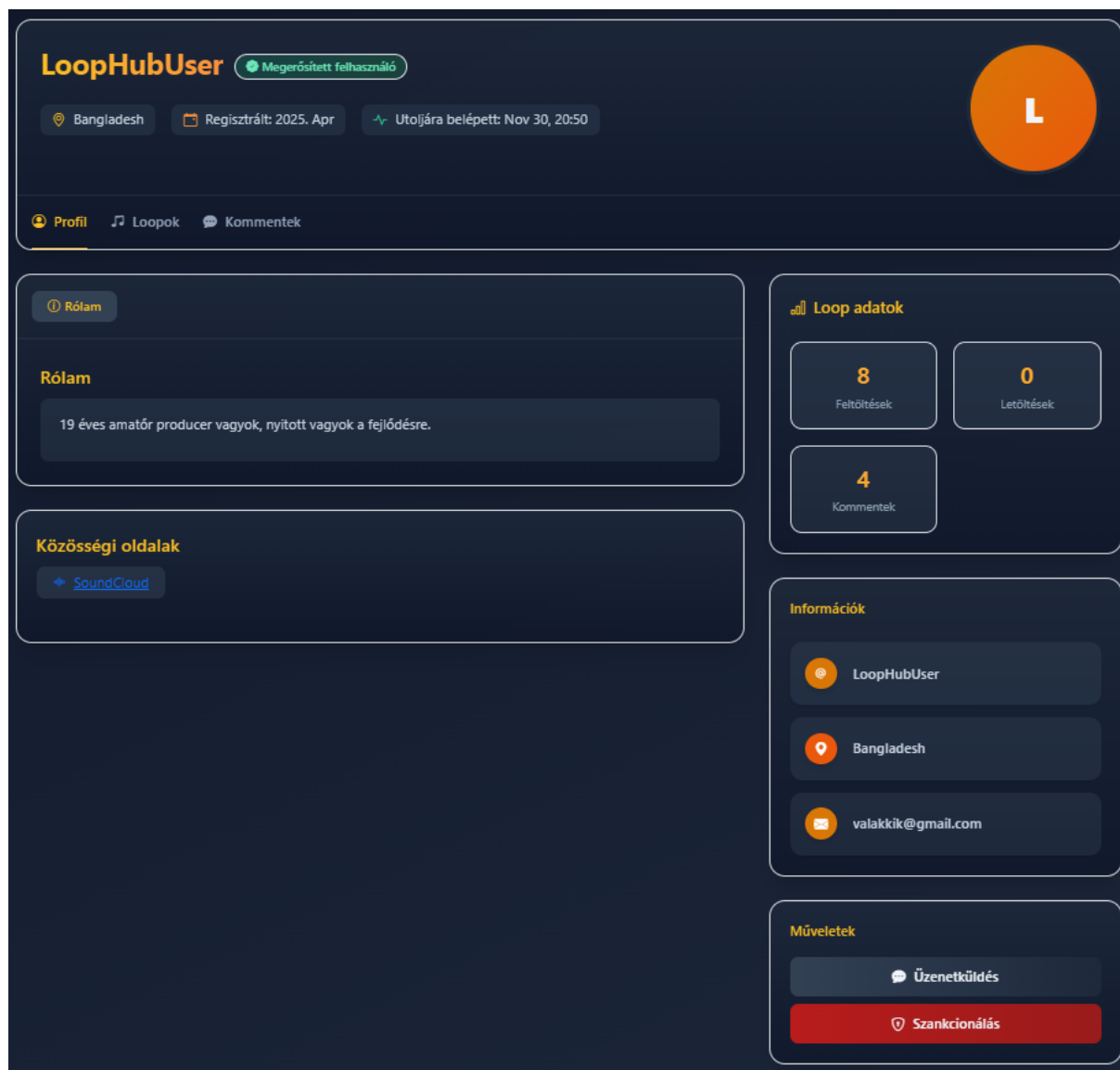
40. ábra. Részletes loop oldal



41. ábra. Hozzászólási lehetőség

4.4.5. Profil

A felhasználói profilon megtalálhatók a regisztráláskor megadott adatok, valamint loopösszefoglalók.



42. ábra. Profil oldal

Az üzenetküldés gombra kattintva megjelenik az üzenetküldő ablak, ahol chatelni lehet a felhasználóval. Ezen üzenet után megjelenik a navigációs sor üzenetablakában.

5. IRODALOMJEGYZÉK

- [1] Matija Turk, Ivan Turk, Marcel Otte: *The Neanderthal Musical Instrument from Divje Babe I Cave (Slovenia): A Critical Review of the Discussion*, 2019
<https://www.mdpi.com/2076-3417/10/4/1226>
- [2] Szabados Éva: *A zene mint erőforrás*, 2025. 05. 27
<https://www.gyoriszaalon.hu/news/17341/66/>
- [3] Christoph Riedweg: *Pythagoras: His Life, Teaching, and Influence*, 2002
https://api.pageplace.de/preview/DT0400.9780801464843_A29967304/preview-9780801464843_A29967304.pdf
- [4] Wikipédia, 2025. 05. 28.
https://hu.wikipedia.org/wiki/Renesz%C3%A1nsz_zene
- [5] Wikipédia, 2025. 05. 28.
https://hu.wikipedia.org/wiki/Barokk_zene
- [6] Franz Xaver Niemetschek: *Mozart: The First Biography*, 2007
<https://www.google.hu/books/edition/Mozart/6yCjBgAAQBAJ?hl=en&gbpv=0>
- [7] Várkonyi Gábor: *Romantika – egy korstílus esztétikai alapjai*, 2025. 05. 30.
<https://varkonyitanarur.hu/korszakok/romantika/>
- [8] Lengyel Erik Krisztián: *Notációs jelek megfejtése Karlheinz Stockhausen Zyklus című műve alapján*, 2025. 06. 30.
https://www.parlando.hu/2019/2019-3/Lengyel_Erik.pdf
- [9] Farkas István Péter: *Rézfúvós hangszerek története Magyarországon*, 2014
<https://www.ibew.org.uk/dvarch/DV01568.pdf>
- [10] *Music Theory Basics Every Music Producer Should Know*, 2024
<https://www.masteringthemix.com/blogs/learn/music-theory-basics-every-music-producer-should-know>
- [11] MasterClass: *How to Write a Melody: 9 Tips for Writing Memorable Melodies*, 2021
<https://www.masterclass.com/articles/how-to-write-a-melody>
- [12] Aden Russell: *How To Make Drum & Bass: The Complete Guide*, 2025
<https://www.edmprod.com/how-to-make-drum-and-bass/>
- [13] Az FL-Studio program hivatalos weboldala:
<https://www.image-line.com/>
- [14] Az Ableton program hivatalos weboldala:
<https://www.ableton.com/>
- [15] A Logic Pro program hivatalos weboldala:
<https://www.apple.com/logic-pro/>

- [16] A MongoDB hivatalos weboldala:
<https://www.mongodb.com/>
- [17] Angular keretrendszer telepítése és beállítása:
<https://code.visualstudio.com/docs/nodejs/angular-tutorial>
- [18] NodeJS futtatókörnyezet telepítése és beállítása:
<https://nodejs.org/en/download/>
- [19] NodeJS megfelelő implementálása Visual Studio Code-ba:
<https://code.visualstudio.com/docs/nodejs/nodejs-tutorial>
- [20] JestJS megfelelő implementálása a különböző keretrendszerekhez:
<https://jestjs.io/docs/testing-frameworks>